



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE ESTUDIOS PROFESIONALES CAMPUS ACATLÁN

MODELOS EXPLICABLES DE MACHINE
LEARNING PARA LA PREDICCIÓN DEL CHURN Y
CALCULO DEL CHURN RATE

T E S I S

QUE PARA OPTAR POR EL GRADO DE:
Licenciado en Matemáticas Aplicadas y Computación

PRESENTA:

Carlos Espadin Medina

TUTOR:

Mtra. Jeanett López García



Ciudad de México, 2024

*A la Facultad de Ingeniería y a la Universidad, por la formación que me han dado.
Es gracias a ustedes que es posible el presente trabajo.
En verdad, gracias.
Yo.*

Reconocimientos

También quisiera reconocer a ... por ...CONACYT, PAPIIT / etc. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Declaración de autenticidad

Por la presente declaro que, salvo cuando se haga referencia específica al trabajo de otras personas, el contenido de esta tesis es original y no se ha presentado total o parcialmente para su consideración para cualquier otro título o grado en esta o cualquier otra Universidad. Esta tesis es resultado de mi propio trabajo y no incluye nada que sea el resultado de algún trabajo realizado en colaboración, salvo que se indique específicamente en el texto.

Carlos Espadin Medina. Ciudad de México, 2024

Resumen

This is where you write your abstract ...

Índice general

Índice de figuras	XI
Índice de tablas	XIII
1. Introducción	1
1.1. Presentación	1
1.2. Objetivo	1
1.3. Motivación	1
1.4. Planteamiento del problema	2
1.5. Preguntas de investigación	2
1.6. Hipótesis o supuestos del estudio	2
1.7. Metodología	2
1.8. Contribuciones	2
1.9. Estructura de la tesis	3
2. Marco teórico y conceptual	5
2.1. Fundamentos del aprendizaje automático (ML)	5
2.2. Aprendizaje Supervisado	8
2.2.1. Algoritmos de clasificación	11
2.2.1.1. Regresión logística	13
2.2.1.2. K Nearest Neighbors	18
2.2.1.3. Random Forest	21
2.2.1.4. SVM	25
2.3. Predicción del churn.	25
2.3.1. Definición de <i>churn</i>	25
2.3.2. Cálculo del <i>churn</i>	25
2.3.3. Métricas y causalidad del <i>churn</i>	28
2.3.4. Análisis de Cohortes y la tasa de abandono	29
2.4. Inteligencia artificial explicable (XAI)	33
2.4.1. SHAP	33
2.4.2. LIME	33
2.4.3. Importancia de características	33

3. Metodología de análisis y preparación de los datos	35
3.1. Descripción del conjunto de datos	35
3.2. Preprocesamiento	36
3.2.1. Limpieza y tratamiento de valores faltantes y outliers	36
3.2.2. Transformaciones y codificación	36
3.2.3. Selección y reducción de características (PCA)	36
3.3. Construcción de la variable de abandono (churn)	36
3.4. Implementación de modelos de ML para predecir el abandono	37
3.4.1. Evaluación del modelo.	37
3.4.2. Comparación del rendimiento entre modelos.	41
3.4.3. IA Explicable.	48
4. Implementación y evaluación de modelos predictivos	51
4.1. Implementación de modelos de clasificación	51
4.1.1. Descripción de cada modelo y sus supuestos	51
4.2. Evaluación de desempeño de los modelos	51
4.2.1. Métricas: Accuracy, Precision, Recall, F1, AUC, PR	51
4.2.2. Comparación de resultados	51
4.3. Aplicación de XAI a modelos seleccionados	51
4.3.1. Interpretación de resultados y visualizaciones	51
5. Discusión y análisis de resultados	53
5.1. Discusión de hallazgos relevantes	53
5.2. Limitaciones del modelo	53
5.3. Recomendaciones para su implementación en entornos empresariales reales	53
6. Conclusiones y líneas futuras	55
6.1. Conclusiones generales	55
6.2. Aportaciones teóricas, metodológicas y prácticas	55
6.3. Sugerencias para investigaciones futuras	55
A. Código/Manuales/Publicaciones	57
A.1. Apéndice	57
Bibliografía	59

Índice de figuras

2.1. El enfoque tradicional	6
2.2. Enfoque de Machine Learning	7
2.3. Relación entre IA, ML, Deep Learning y tipos de aprendizaje automático	10
2.4. Tipos de aprendizaje automático con ejemplos de tareas y algoritmos . .	11
2.5. Función Sigmoide.	13
2.6. La categoría objetivo del punto ? varía con el número de nodos más adyacentes.	18
2.7. Ejemplo de árbol de decisión (con posicionamiento manual) para la pre- dicción de Churn.	21
2.8. Histograma de Tiempo desde la ultima compra	27
2.9. Gráfica de tarda para el abandono de clientes.	28
2.10. Matriz de Cohortes.	34
3.1. Matriz de confusión para el modelo de multi-layer perceptron classifier.	37
3.2. Curva ROC.	40
3.3. Curva PR.	41
3.4. Matriz de confusión para el modelo de regresión logística del conjunto de entrenamiento y validación	42
3.5. Curva PR de Regresión Logística	42
3.6. Metricas de evaluación del modelo de regresión logística para el conjunto de entrenamiento y validación	43
3.7. Matriz de confusión para el modelo KNN del conjunto de entrenamiento y validación	43
3.8. Metricas de evaluación del modelo de KNN para el conjunto de entrena- miento y validación	44
3.9. Matriz de confusión para el modelo de soport vector machine del con- junto de entrenamiento y validación	45
3.10. Metricas de evaluación del modelo de SVM para el conjunto de entrena- miento y validación	45
3.11. Matriz de confusión para el modelo random forest del conjunto de en- trenamiento y validación	46
3.12. Metricas de evaluación del modelo de random forest para el conjunto de entrenamiento y validación	47

ÍNDICE DE FIGURAS

3.13. Curvas Roc de los modelos de clasificación para el conjunto de entrenamiento y validación respectivamente.	47
3.14. Gráfica de resumen global	48
3.15. Gráfica de explicación local	48
3.16. Gráfica de decisiones	49

Índice de tablas

Introducción

1.1. Presentación

1.2. Objetivo

El principal objetivo de este trabajo es desarrollar, analizar y modelar el comportamiento de los clientes de cualquier empresa en México, el enfoque de este trabajo puede ser dirigido tanto a empresas que ofrezcan servicios de retail, suscripciones, telecomunicaciones, financieros/bancarios o inclusive empresas de nicho como las pertenecientes al sector agrícola, todo esto utilizando técnicas de aprendizaje automático, más en específico usaremos modelos de clasificación para predecir su comportamiento, la idea es identificar patrones de conducta asociados a la retención y al abandono de clientes (Churn), haciendo uso de herramientas de ciencia de datos implementadas en Python. La finalidad es permitir a cualquier tipo de empresas anticipar las posibles pérdidas de clientes y diseñar diversas estrategias de fidelización.

1.3. Motivación

Popularmente se cree que adquirir nuevos clientes cuesta más que mantener los actuales, tal afirmación plantea la importancia que toma el análisis de churn en la toma de decisiones. Hoy en día, la retención de clientes representa uno de los mayores desafíos para las empresas en todo tipo de sectores, este caso de estudio se ha aplicado a empresas de retail, telecomunicaciones, servicios financieros, entre otros. La innovación científica y tecnológica en las empresas de México y todo el mundo avanza a pasos agigantados y con ella surgen distintas necesidades, el poder incorporar herramientas de análisis de datos para poder comprender mejor el comportamiento de sus clientes es una de ellas, en especial a la hora de comprender las razones por las cuales sus clientes dejan de consumir sus servicios/productos, por lo cual el análisis de abandono

1. INTRODUCCIÓN

de clientes se ha convertido en un requisito estratégico. Esta investigación surge por la motivación de integrar un enfoque cuantitativo, dirigido al Machine Learning, esto con un enfoque científico para poder ofrecer una solución innovadora a todo tipo sectores, incluso aquellos donde la digitalización aun esta en desarrollo.

1.4. Planteamiento del problema

El objetivo será desarrollar un modelo de explicativo y predictivo del Churn utilizando diferentes algoritmos de Machine Learning, comparando tanto modelos clásicos (por ejemplo regresión logística, maquinas de soporte vectorial, entre otros) así como modelos más complejos (XGBoost, redes neuronales, etc.), de tal manera que podamos estimar la capacidad de estos modelos para predecir el Churn y explicar el porqué ciertos clientes tienen más probabilidad de abandonar. La importancia de ofrecer una buena alternativa que explique y prediga dicho fenómeno ayudaría a poder generar hallazgos y accionables.

1.5. Preguntas de investigación

1.6. Hipótesis o supuestos del estudio

1.7. Metodología

La metodología propuesta para el desarrollo de este trabajo esta constituida por cuatro etapas primordiales, **primero** recopilación y preprocesamiento de los datos, **segundo** implementación de modelos de aprendizaje automático (Machine Learning y Deep Learning), **tercero** aplicación de técnicas de inteligencia artificial explicable (XAI) y finalmente en **cuarto** lugar la evaluación comparativa de los modelos. Dicha estructura nos va a permitir ofrecer un análisis predictivo y explicativo del Churn, de tal suerte que la comprensión e interpretación de los factores que explican el abandono de clientes se puedan aplicar de manera transversal a diferentes sectores de la iniciativa privada.

1.8. Contribuciones

Este trabajo plantea una serie de aportaciones tanto teóricas como prácticas en el campo de la ciencia de datos enfocados al Churn. Las principales aportaciones se describen a continuación:

1. Diseño de un pipeline generalizado y modular para la predicción del churn mediante aprendizaje automático.
2. Comparación sistemática entre modelos clásicos y avanzados de Machine Learning, integrando criterios de precisión y explicabilidad.
3. Aplicación práctica orientada a la toma de decisiones empresariales.
4. Contribución al debate sobre la explicabilidad en sistemas de inteligencia artificial aplicados a negocios.

1.9. Estructura de la tesis

Este trabajo está dividido en 5 capítulos, todos y cada uno de ellos responde a una fase del proceso de esta investigación, las cuales van desde la contextualización del problema (Churn) hasta el análisis de los resultados y las conclusiones. A continuación, describiremos brevemente el contenido de cada capítulo:

1. Introducción
2. Revisión de la literatura
3. Metodología
4. Resultados
5. Conclusiones

Marco teórico y conceptual

2.1. Fundamentos del aprendizaje automático (ML)

En los últimos años, la IA se ha vuelto parte de la cultura popular de nuestra sociedad y temas como el aprendizaje automático, ha alcanzado un hito en popularidad bastante marcado, esto debido a dentro de la misma inteligencia artificial esta es una de las ramas más destacadas, permitiendo a un sistema aprender de manera automática patrones a partir de datos y con los mismos poder realizar predicciones precisas a partir de los datos dados. En pocas palabras, el Machine Learning como nos referiremos al aprendizaje automático de ahora en adelante, busca que los sistemas informáticos “aprendan” de las experiencias de los datos, esto sin ser programados explícitamente para llevar a cabo una tarea en específico.

Sin embargo, pese a la reciente popularidad de la inteligencia artificial, que de ahora en adelante nos referiremos a ella como IA, y el creciente interés del público en general por todos los temas concernientes a la misma, es un hecho que desde mediados del siglo pasado se comenzó a gestar el desarrollo de la IA, y desde inicios de los 2000 diversas y discretas aplicaciones de la IA acompañaron la vida de las personas, desde integraciones en video juegos hasta motores de búsqueda como Google.

Una de las definiciones que mejor describe el machine learning es la citada por [Mitchell \(1997\)](#):

A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

—*Tom Mitchell, 1997*

A manera de ejemplo para poder entender como funciona el Machine Learning ([Géron, 2017](#), p. 4) explica esta definición usando como ejemplo el filtro de correos

2. MARCO TEÓRICO Y CONCEPTUAL

electrónicos de spam, la cual es una aplicación del machine learning, dice que el sistema puede aprender a marcar spam a partir de ejemplos de correos electrónicos spam (por ejemplo, marcados por los usuarios) y ejemplos de correos electrónicos normales (no spam, también llamados "ham"). Los ejemplos que el sistema utiliza para aprender se denominan conjunto de entrenamiento. Cada ejemplo de entrenamiento se denomina instancia de entrenamiento (o muestra). En este caso, la tarea T consiste en marcar el spam para los correos electrónicos nuevos, la experiencia E son los datos de entrenamiento y es necesario definir la medida de rendimiento P ; por ejemplo, se puede utilizar la proporción de correos electrónicos correctamente clasificados.

Esta explicación se puede extender de igual forma al problema central de esta tesis, el churn, pensemos que tradicionalmente una empresa tiene ciertas reglas de negocio para considerar a un cliente como perdido o Churn, más tarde ahondaremos en las posibles definiciones de dichas reglas, por el momento supongamos que una empresa define n cantidad de reglas para clasificar a sus clientes como churn y no churn, si dichas reglas se trasladan a un software el resultado sería algo muy similar al diagrama 2.1.

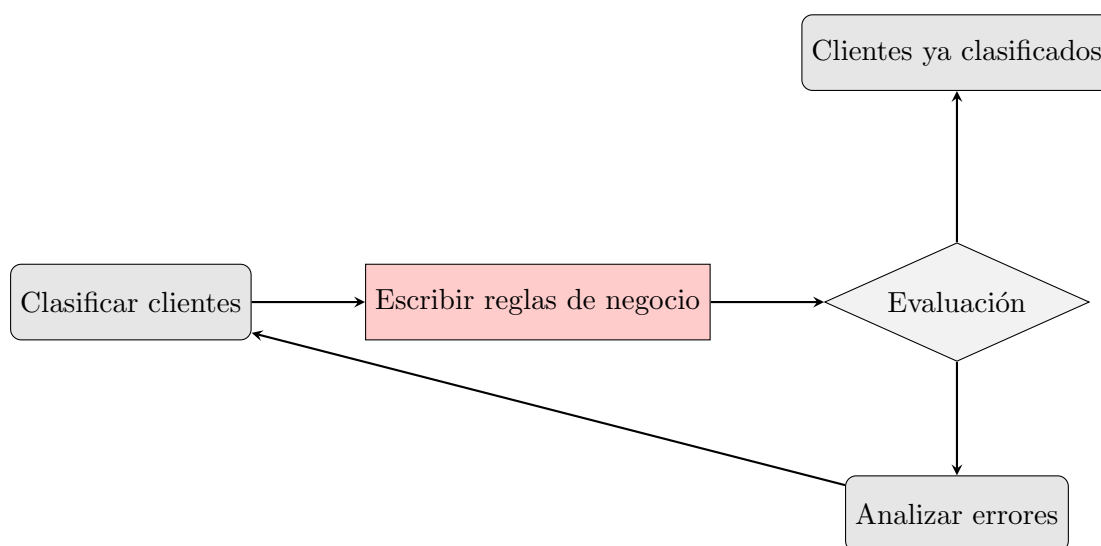


Figura 2.1: El enfoque tradicional

El problema de este enfoque tradicional es claro, una vez que se identifiquen las áreas de oportunidad en las reglas de negocio se tendrían que hacer adecuaciones en las mismas, quitar o añadir nuevas, principalmente esto ultimo paso va consumir mucho tiempo en mantenimiento y aumenta enormemente la complejidad de dichas validaciones, ya que por cada error cometido por el sistema se tendrían que estar añadiendo más y más reglas de negocio para obtener clasificaciones lo más certeras.

En cambio, si esta misma tarea la realizamos usando un enfoque de Machine Learning la trazabilidad cambia totalmente, ya que el algoritmo de ML (Machine Learning) aprende de manera automática las caracterizaras que son buenos predicadores de clientes churn y no churn, al igual que el sistema esta mucho más acotado, es más fácil de

entender y, muy probablemente, sea más preciso. El como abordar el Machine Learning este problema lo podemos ver más claramente en el diagrama 2.2, donde notamos que en esta ocasión en flujo del proceso ya no depende de reglas de negocio establecidas manualmente, esta vez son los datos históricos quienes alimentan al modelo. Este aprende por sí solo las relaciones complejas entre las variables que describen a los clientes, permitiendo generalizar a nuevos casos con mayor precisión.

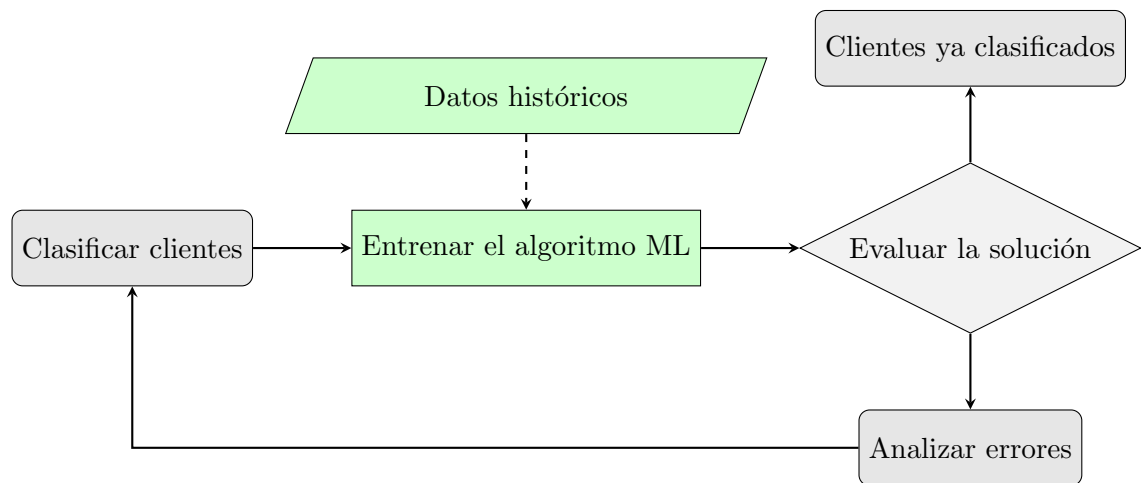


Figura 2.2: Enfoque de Machine Learning

En este enfoque, los datos se convierten en el suministro más valioso, una vez que nuestro modelos este entrenado, se puede evaluar su rendimiento utilizando métricas objetivas del rendimiento, como la exactitud, la precisión, etc. Si el rendimiento no es el esperado, se pueden analizar los fallos, ajustar el modelo o incluso recolectar nuevos datos, para así reiniciar el bucle de aprendizaje.

Algunas de las ventajas del enfoque de Machine Learning son la *escalabilidad*, ya que los modelos pueden adaptarse sencillamente a nuevos datos sin tener que rescribir reglas de negocio de forma manual. Otra ventaja es **capacidad para detectar patrones complejos**, esto ya que los algoritmos de pueden reconocer correlaciones no lineales y dependencias que a simple vista sería muy difíciles de identificar mediante validaciones fijas. Por ultimo y no menos importante, **la capacidad de mejorar continuamente** el ciclo iterativo nos permite evolucionar progresivamente nuestro sistema esto conforme van cambiando ciertos patrones de comportamiento de los clientes. De tal suerte que aplicar modelos de Machine Learning en la predicción del churn no solo representa una mejora tecnológicamente hablando, sino también un cambio de paradigma en la manera en que las empresas pueden combatir el abandono de clientes.

2.2. Aprendizaje Supervisado

Ya hemos abordado una pequeña introducción al aprendizaje automático o machine learning, y como usando esta perspectiva podemos ayudarnos para resolver distintos problemas de la vida cotidiana en múltiples ámbitos, entre ellos el empresarial, además su implementación puede ser más práctica, escalable y fácil de mantener a lo largo del tiempo. Sin embargo, es necesario saber que el machine learning es un tipo de inteligencia artificial, y aun que anteriormente hemos mencionado con cierta naturalidad el concepto de inteligencia artificial es necesario dar un par de definiciones para entender mejor este concepto, según [Russell and Norvig, 2020](#):

”La inteligencia artificial es el estudio de los agentes que reciben percepciones del entorno y realizan acciones.”

[Russell and Norvig \(2020, p. 1\)](#)

Otra definición bastante interesante es la dada por [Luger, 2005](#):

”La inteligencia artificial es el estudio de los mecanismos computacionales para la percepción y el razonamiento, que pueden imitar la inteligencia humana.”

[Luger \(2005\)](#)

De tal forma que si hilamos estas definiciones con la antes dada para el machine learning, podemos darnos cuenta que es un caso particular de la inteligencia artificial, ya que el machine learning busca brindar a las máquinas la capacidad de aprender a partir de datos, y aunque el propósito no sea emular el razonamiento humano, la importancia del machine learning reside en poder capturar patrones y predecir resultados futuros con cierta medida de la incertidumbre. Este mismo tiene varios subtipos o ramas, las cuales son:

- **Aprendizaje supervisado**
- **Aprendizaje no supervisado**
- **Aprendizaje semi-supervisado**
- **Aprendizaje por refuerzo**
- **Aprendizaje por lotes y en línea**

Y de la misma forma en que el machine learning hace parte de la inteligencia artificial como una rama dentro de esta disciplina, así como todos los tipos de aprendizaje automático que mencionamos anteriormente, el diagrama 2.3 ejemplifica perfectamente la jerarquía entre inteligencia artificial, machine learning e incluso el deep learning o aprendizaje profundo que es un tipo de aprendizaje automático, una de las definiciones más famosas es la siguiente:

”Deep learning is a class of machine learning algorithms that use multiple layers to progressively extract higher-level features from the raw input.”

LeCun et al. (2015)

De hecho uno de los diferenciadores principales entre el deep learning y los demás tipos de aprendizaje automático es la que se menciona en la definición anterior, los modelos de deep learning, los cuales suelen ser Redes neuronales, minimizan la intervención humana a la hora de selección de características, lo que permite una mayor automatización y eficiencia en problemas de complejidad mayor.

Podríamos continuar hablando acerca de todas las ramas en las que se especializa la inteligencia artificial sin embargo no es el propósito de este trabajo. Pero algo que si esta dentro del alcance de este trabajo es el aprendizaje supervisado, el cual es una de las ramas más importantes dentro del aprendizaje automático y la base principal para elaborar modelos predictivos como los que podríamos usar para predecir y explicar el churn.

En términos generales este enfoque es muy parecido al ejemplo del diagrama 2.2, entrenamos nuestro modelo (sistema) con datos históricos como ejemplos de entrada, cada uno de los cuales esta asociado a una salida esperada. El objetivo es que este modelo aprenda las distintas relaciones que pudieran existir entre las diferentes variables que se le presenten estas asociadas a un resultado, y este mismo sea capaz de predecir salidas de forma correcta a datos sin un resultado definido.

En términos matemáticos el sistema buscar aprender una función que explique la relación entre las variables de entrada, el cual común mente es llamado matriz de características o variables explicativas, y la variable objetivo. Este proceso puede expresarse de las siguiente manera:

$$f : \mathbb{R}^{n \times m} \rightarrow \mathbb{R}^n \quad (2.1)$$

Donde $\mathbb{R}^{n \times m}$ representa la matriz de características con n observaciones o filas y m características o columnas, por otra parte $\mathbb{R}^n$ es la variable objetivo o termino dependiente, este mismo es el vector de los valores de salida esperados de orden n , además nos referiremos a f como ”modelo”.

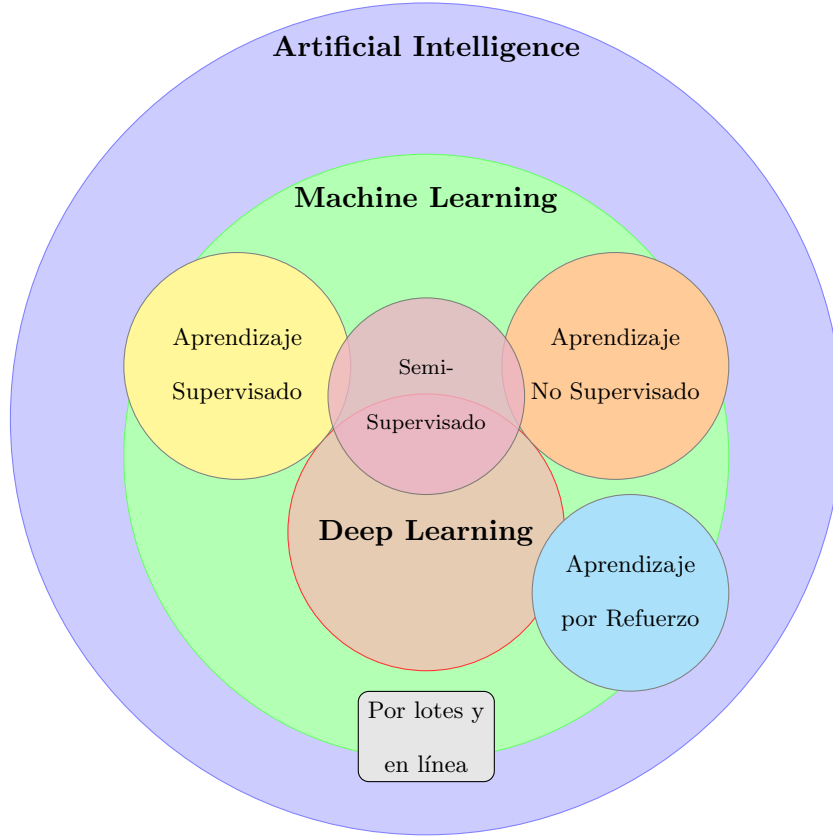


Figura 2.3: Relación entre IA, ML, Deep Learning y tipos de aprendizaje automático

En el caso del Churn cada fila de la matriz $n \times m$ es un cliente, por lo cual tendríamos n clientes con m características, las cuales pueden ser consumo, frecuencia de compra, etc. Similarmente las salidas nos indican si un cliente abandono o no, lo cual se puede representar como un vector $\mathbf{y} = [y_1 \ y_2 \ \dots \ y_n]$, con $v_i \in \{0, 1\}, \forall i \in 1, 2, \dots, n$.

El aprendizaje supervisado resuelve distintos tipos de problemas entre los cuales se encuentra los de tipo regresión y clasificación, mismos que podemos ver ilustrados en el diagrama 2.4, en esta tesis solo abordaremos en profundidad los problemas de clasificación ya que el churn puede reducirse a eso.

El diagrama 2.4 nos muestra que los problemas de regresión, clasificación o segmentación (que son un tipo de aprendizaje no supervisado) no son exclusivos del aprendizaje supervisado o no supervisado, sino que también pueden ser resueltos con ayuda de redes neuronales, todo depende de la cantidad de datos que tengamos a nuestra disposición o el nivel de intervención para selección de características.

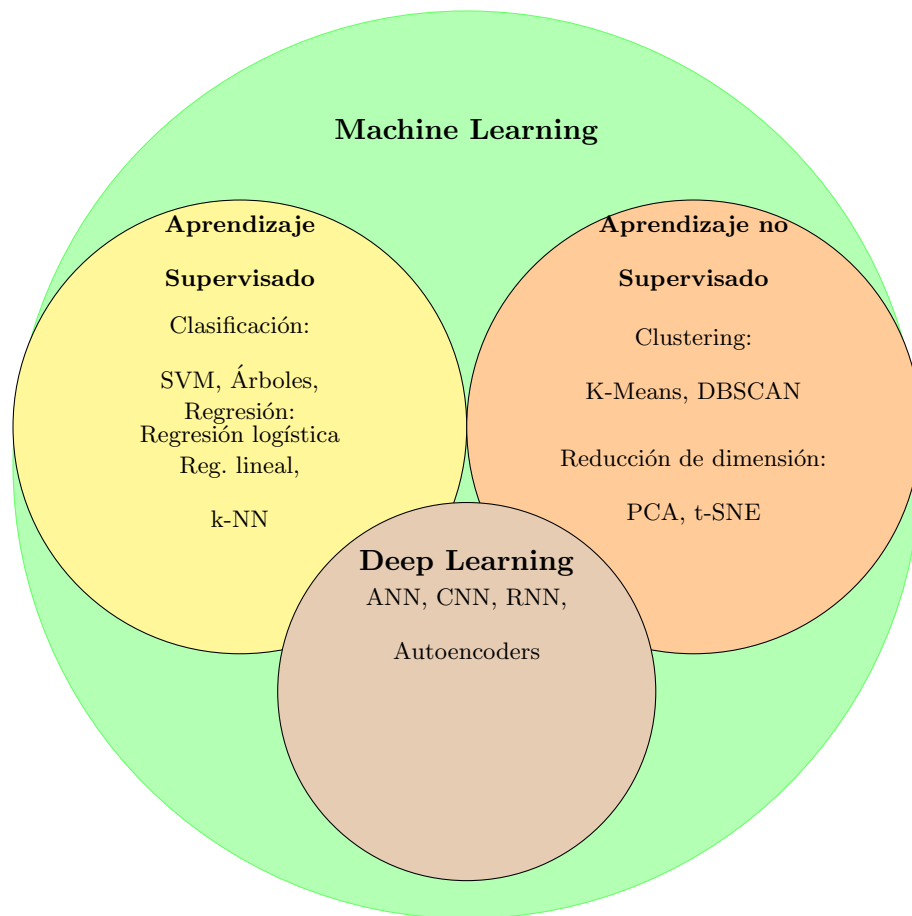


Figura 2.4: Tipos de aprendizaje automático con ejemplos de tareas y algoritmos

2.2.1. Algoritmos de clasificación

El churn se puede reducir a un problema de clasificación, más en específico a una clasificación binaria. Los algoritmos de clasificación permiten predecir la pertenencia de una observación a una categoría o clase definida. En el contexto del churn, el objetivo es predecir si un cliente se mantendrá activo o si abandonará la empresa en un periodo determinado.

En el entorno del Machine Learning, un algoritmos de clasificación son esenciales para poder resolver problemas donde el objetivo es asignar una etiqueta o categoría a un conjunto de datos a partir de las características o variables explicativas. Estos algoritmos construyen modelos predictivos que aprenden patrones a partir de ejemplos históricos que ya está etiquetados, a este conjunto de datos solemos llamarlo conjunto de entrenamiento, para después poder clasificar nuevos registros con base en los datos ya aprendidos. Retomando un poco el problema del abandono de cliente, el objetivo es predecir si un cliente continuara o no en una empresa, con lo que se puede representar

2. MARCO TEÓRICO Y CONCEPTUAL

como una variable binaria, puede ser 1 para los clientes que abandonan y 0 para los cliente que permanecen.

Existen varios algoritmos de clasificación, cada uno con sus defectos y virtudes, el qué algoritmo escoger depende de múltiples factores entre ellos la naturaleza de los datos, que interpretabilidad queramos darle a nuestro problema e inclusive el costo computacional. En este trabajo vamos a considerar cuatro algoritmos clásicos en el aprendizaje automático y más en específico en los problemas de clasificación.

Con base a [Géron \(2017\)](#) enuncio el siguiente teorema, que explica lo anterior:

Teorema No-Free-Lunch

Sea $\mathcal{F}$ el conjunto de todas las funciones posibles entre un espacio de entrada $\mathcal{X}$ y un conjunto de salidas $\mathcal{Y}$. Si se considera un conjunto $\mathcal{A}$ de algoritmos de aprendizaje supervisado o de optimización que operan sobre $\mathcal{F}$, entonces:

$$\frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \text{Performance}(A_i, f) = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \text{Performance}(A_j, f) \quad \forall A_i, A_j \in \mathcal{A}$$

Es decir, el rendimiento promedio de todos los algoritmos es el mismo cuando se consideran todas las funciones posibles. No existe un algoritmo universalmente mejor sin asumir algún conocimiento previo (sesgo inductivo) sobre el problema.

Este teorema trae una serie de insinuaciones profundas, el hecho de que no exista un modelo de clasificación “ideal”, esto nos obliga a realizar una serie de validaciones rigurosas de forma práctica para determinar el mejor modelo, osea el que se acople mejor a nuestro conjunto de datos en particular. En nuestro caso, para el churn esto se traduce en que diferentes dataset, como los que tengan granularidad por sector, región o productos, etc., pueden necesitar distintos modelos para lograr una buena generalización.

A continuación, se abordara la teoría de los algoritmos utilizados en la predicción de churn, vamos a poner especial foco en sus fundamentos, ventajas, limitaciones y su interpretabilidad. Es importante remarcar que en este trabajo abordaremos el problema de clasificación binaria, el cual es un caso particular de la clasificación múltiple.

2.2.1.1. Regresión logística

La regresión logística es uno de los modelos más clásicos y popularmente usados en la predicción del churn. Este algoritmo nos ayuda a clasificar las observaciones de nuestro conjunto de datos en dos o más categorías. Básicamente deseamos determinar una regla de decisión a partir de n variables y asignar una respuesta que puede tomar dos o más valores. Más aun tomando en cuenta el mapa (2.2) podemos derivar la siguiente función:

$$f : x \in \mathbf{R}^{n \times m} \rightarrow y^n \in \{0, 1\}$$

Lo anterior explica la relación que determinaremos con ayuda de los algoritmos de clasificación binaria en general, incluyendo a la regresión logística y los demás modelos que abordaremos en esta tesis. Aun que antes de entrar en materia con la función sigmoide y la regresión logística conviene mencionar retomar un concepto que de hecho es una generalización de la misma regresión logística, este es la regresión lineal simple, osea vamos a asumir que solo tendremos una variable predictora. A continuación enunciamos la definición del modelo de regresión lineal simple.

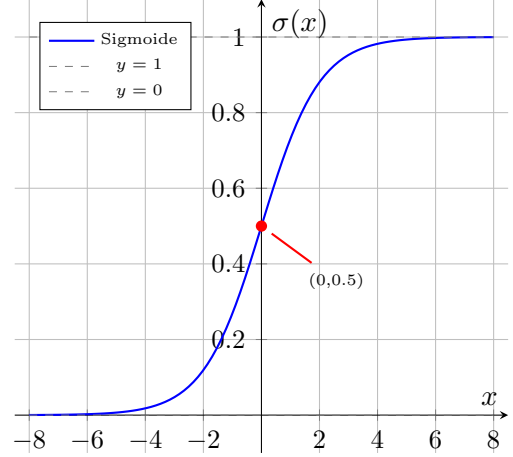


Figura 2.5: Función Sigmoide.

Definición 2.2.1: Definición: Regresión Lineal simple

Un modelo estadístico lineal que relaciona una respuesta aleatoria Y con una variable independiente x es de la forma:

$$Y = \beta_0 + \beta_1 x_1 + \varepsilon \quad (2.2)$$

Además supondremos que $\mathbb{E}[\varepsilon] = 0$ y por tanto:

$$\mathbb{E}[Y|x] = \beta_0 + \beta_1 x \quad (2.3)$$

donde:

- $\beta_0 \in \mathbb{R}$ es el intercepto (o término independiente).
- $\beta_1 \in \mathbb{R}$ es el coeficiente de x .
- $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ representa un término de error aleatorio.

La regresión lineal al igual que la covarianza y el coeficiente de correlación (lineal) de pearson, nos ayudan a determinar y analizar la relación entre dos variables, sin embargo la regresión lineal va más allá porque nos permite medir el grado de asociación entre dos variables, nos permite imitar un modelo que describa la relación entre estas

2. MARCO TEÓRICO Y CONCEPTUAL

variables, poder predecir el comportamiento de una de ellas a partir de valores de la otra y hacer mejores inferencias sobre la variable de interés. Pero la importancia de la regresión lineal, en esta tesis, reside en que con ayuda de esta podemos deducir la función logística, y es que ambos comparten un núcleo en común, la ecuación $\beta_0 + \beta_1 x$, aun que la diferencia radica en como la regresión logística adapta brillantemente esta ecuación, lo conecta a través de la transformación "sigmoide", lo cual le permite usar el poder y la simplicidad de un modelo lineal y aplicarlo para resolver un problema de clasificación, a continuación la definición del modelo.

Definición 2.2.2: Regresión Logística Binaria simple

Dado un conjunto de datos $\mathcal{D}$ con N pares de entrada y etiqueta binaria (x_n, y_n) . El modelo de regresión logística es un modelo lineal generalizado que relaciona la probabilidad de ocurrencia de un evento binario Y con un predictor $x \in \mathbb{R}^n$ mediante la siguiente ecuación:

$$\mathbb{P}(Y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

o equivalentemente, en términos del logit (log-odds):

$$\ln \left(\frac{\mathbb{P}(Y = 1 \mid x)}{1 - \mathbb{P}(Y = 1)} \right) = \beta_0 + \beta_1 x$$

Aquí, β_0, β_1 son los coeficientes del modelo. La función logística transforma la combinación lineal de los predictores en una probabilidad dentro del intervalo $[0, 1]$.

De forma ilustrativa vamos a "demostrar" el modelo logístico a partir de la ecuación logística. El objetivo es modelar la probabilidad de una variable de respuesta binaria, osea la probabilidad de que $Y = 1$ dado un predictivo x , a esta probabilidad la denotaremos como $\mathbb{P}(Y = 1 \mid x) = p(x) \in [0, 1]^n$. Si tomamos en consideración la igualdad (2.2), la idea es determinar una función $L(x)$ tal que:

$$L(x) = \beta_0 + \beta_1 x \tag{2.4}$$

Sin embargo, podemos notar que $(\beta_0 + \beta_1 x) \in \mathbb{R}^n$, lo cual contradice el rango que necesitamos que tome $p(x)$. Por otra parte si consideramos a (2.3), el cual suele ser llamado modelo ajustado o determinista, podemos notar que es exactamente igual a (2.4). Por otra parte, un resultado que tenemos que tener en cuenta es la función sigmoide, la cual esta definida de la siguiente manera:

$$\sigma(t) = \frac{1}{1 + e^{-t}} = \frac{e^t}{1 + e^t} \tag{2.5}$$

Ahora bien, si aplicamos una composición de funciones ρL de tal forma que:

$$\begin{aligned} p(x) &= \sigma(L(x)) \\ &= \sigma(\beta_0 + \beta_1 x) \\ &= \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}} \end{aligned}$$

De esta manera podemos ver la intrínseca relación que existe entre la regresión lineal y la logística, por lo cual este último hereda algunos supuestos y propiedades de la regresión lineal, las cuales son:

1. **Independencia.** Las observaciones son independientes entre sí
2. **Relación lineal entre los predictores y el LOGIT del resultado.** Se asume que la transformación logit de la probabilidad es una función lineal de las variables predictoras.
3. **Ausencia de multicolinealidad.** Las variables predictoras (las "x") no deben estar altamente correlacionadas entre sí.
4. **Tamaño de muestra suficientemente grande.** Si la regresión logística se basa en la estimación por máxima verosimilitud, cuyas propiedades (como la normalidad de los coeficientes) son asintóticas, es decir, se cumplen mejor con muestras grandes. Una regla general es tener al menos 10 eventos (el resultado menos frecuente) por cada variable predictora que se incluya en el modelo.

Otro concepto que resulta crucial mencionar es la función de pérdida, este concepto está íntimamente relacionado a uno que suele usarse mucho en el campo de la probabilidad, este es la entropía, que en pocas palabras es la medida cuantitativa de la indeterminación de cualquier variable aleatoria con un número finito de resultados posibles. En nuestro caso nos apoyamos de este concepto para hacer una evaluación del modelo y poder determinar que tan acertadas fueron las clasificaciones con respecto a las observaciones reales, en ocasiones se le llama función de pérdida pero también es conocida como función de entropía cruzada binaria, esto nos permite saber que tan ajustado o no está nuestro modelo a los datos de entrenamiento y asegurar que nuestro modelo generalice bien, en otras palabras es nuestra principal arma para regularizar el sobre ajuste. La función de pérdida de la regresión logística se puede estimar haciendo uso de la estimación por máxima verosimilitud, a continuación mostraremos la definición de dicha función:

Definición 2.2.3: Función de Costo con Entropía Cruzada Regularizada

Dado un conjunto de datos $\mathcal{D}$ con N pares de entrada y etiqueta binaria (x_n, y_n) , donde $y_n \in [0, 1]$, y un modelo que predice la probabilidad $\hat{y}_n = \sigma(x_n)$, la función de costo regularizada $J(\beta)$ que se minimiza para encontrar los coeficientes óptimos del modelo beta se define como:

$$J(\beta) = \left(\frac{1}{N} \sum_{n=1}^N l(y_n, \hat{y}_n) \right) + \lambda \cdot R(\beta) \quad (2.6)$$

donde el primer término es la *pérdida de entropía cruzada promedio* y el segundo es el *término de regularización*. La pérdida individual $l(y, \hat{y})$ sigue siendo:

$$l(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (2.7)$$

El término de regularización $R(\beta)$ depende del tipo de penalización elegida:

- **Penalización L2 (Ridge):** $R(\beta) = \frac{1}{2} \sum_{j=1}^m \beta_j^2$
- **Penalización L1 (Lasso):** $R(\beta) = \sum_{j=1}^m |\beta_j|$

Si bien la función de pérdida de entropía cruzada por si sola nos ayuda a construir mejores modelos y evitar el sobre ajuste, más adelante hablaremos más acerca de este concepto, una interpretación que podemos darle es que esta ecuación nos permite encapsular los dos escenarios posibles:

- Cuando la clase real es **positiva** ($y = 1$), el segundo término se anula ($1 - y = 0$), y la pérdida se simplifica a: $l(1, \hat{y}) = -\log(\hat{y})$.
- Cuando la clase real es **negativa** ($y = 0$), el primer término se anula ($y = 0$), y la pérdida se simplifica a: $l(0, \hat{y}) = -\log(1 - \hat{y})$.

De tal forma que la fórmula nos ayuda medir la discrepancia entre la probabilidad predicha y la etiqueta verdadera, activando el término correspondiente para cada caso. Ahora bien, en la práctica no siempre es conveniente minimizar esta función directamente, lo ideal sería no minimizar la función de pérdida directamente, sino hacer uso de una función de costo regularizada y términos de regularización.

La fórmula (2.6) nos permite incitar a nuestro modelo a ser más preciso, mientras que el término de regularización $R(\beta)$ lo empuja a ser más simple al penalizarlo los coeficientes más grandes.

Sabiendo esto, la verdadera magia ocurre cuando generalizamos este modelo para añadir más características, esta generalización del modelo es clave para resolver más problemas de la vida real, como el churn, donde el comportamiento de un cliente rara vez se explica por un único factor. La definición de este modelo es la siguiente:

Definición 2.2.4: Regresión Logística Binaria múltiple

El modelo de regresión logística es un modelo lineal generalizado que relaciona la probabilidad de ocurrencia de un evento binario $Y \in \{0, 1\}$ con un conjunto de variables independientes $x_1, x_2, \dots, x_k$ mediante la siguiente ecuación:

$$\mathbb{P}(Y = 1 \mid x_1, x_2, \dots, x_k) = \frac{1}{1 + \exp(-(\beta_0 + \beta_1 x_1 + \dots + \beta_k x_k))}$$

o equivalentemente, en términos del logit (log-odds):

$$\log\left(\frac{\mathbb{P}(Y = 1)}{1 - \mathbb{P}(Y = 1)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

Aquí, $\beta_0, \beta_1, \dots, \beta_k$ son los coeficientes del modelo. La función logística transforma la combinación lineal de los predictores en una probabilidad dentro del intervalo $(0, 1)$.

Entre otras cosas algunos de los aspectos más valiosos de la regresión logística, y una de las principales causas de su popularidad en diversas disciplinas e industrias al rededor del mundo, es la capacidad para interpretar sus resultados, y sobre todos los coeficientes del mismo. Esto a diferencia de modelos más complejos, nos permite darle una interpretación más directa y versátil. Por ejemplo el coeficiente β_k con $k = 0, 1, 2, \dots$ se puede interpretar como una escala de crecimiento aditivo, si x_k aumenta en una unidad, se suma β_k al logit, esto si se mantienen contrastantes las demás variables x_j con $j \neq k$, por otra parte el OR (Odds Ratio), que se calcula como e^{β_k} , si x aumenta en una unidad, se multiplica los odds por el $OR(e^{\beta_k})$.

Según [James et al. \(2013\)](#) la estimación de los coeficientes β_k se hace mediante el método de Máxima Verosimilitud o por su abreviación en inglés MLE (Maximum Likelihood Estimation), el objetivo de este estimador es encontrar los valores de los coeficientes que maximicen la probabilidad de haber observado los datos de entrenamiento tal cual son.

Pese a la excelente explicabilidad del modelo de regresión logística, una de sus principales limitaciones es el supuesto de linealidad del logit, se asume que la frontera de decisión que asigna a una clase u otra es lineal. Sin embargo, no siempre se cumple directamente este supuesto en problemas del mundo real, en especial el churn, aun que se pueden aplicar transformaciones previas para tratar de cumplir este supuesto, podemos apelar al uso de otros algoritmos que nos brinden mayor flexibilidad.

2.2.1.2. K Nearest Neighbors

Este es un algoritmo perteneciente al dominio del aprendizaje automático, más específico, aprendizaje supervisado. Se dice que este algoritmo es un clasificador no paramétrico, es decir, no hace suposiciones explícitas sobre la forma funcional de los datos, evitando modelar mal la distribución subyacente de los datos.

De hecho el algoritmo hace un aprendizaje basado en instancias, a veces llamado “aprendizaje perezoso” o lazy learning, esto quiere decir que, el algoritmo no aprende parámetros a partir de los datos de entrenamiento, en cambio memoriza las instancias de formación que posteriormente se utilizan como “conocimiento” para la fase predictiva. Esto contrasta con otros tipos de aprendizaje que van construyendo el modelo de clasificación durante la fase de entrenamiento y posteriormente descartan los datos con los que se entrenó el modelo, esto una vez que el modelo está listo, a esto se le suele llamar “aprendizaje ansioso” o eager learners y algunos de sus principales exponentes son los modelos basados en redes neuronales o support vector machines

Hay un par de frases que describe a la perfección el modelo “el que con lobos anda, a aullar se enseña” o “dime con quién andas y te diré quién eres”, el principio es sencillo, el algoritmo busca en las cercanías, o espacio, los puntos “K” con mayor proximidad de un nuevo dato (de ahí el nombre K vecinos), tales que una vez contados el número de puntos asignados a cada clase asigna el nuevo dato a la categoría con más vecinos en ella. Por ejemplo si $K=5$ y 3 de los 5 vecinos más cercanos son clientes “Churn”, el nuevo cliente será clasificado como “Churn”. A continuación mostraremos el algoritmo del modelo de K Nearest Neighbors. Dado un nuevo punto de datos $\mathbf{x}_{new}$ que se desea clasificar, un conjunto de entrenamiento $(\mathbf{X}_{train}, \mathbf{y}_{train})$, un número entero K y una métrica de distancia d:

1. **Calcular distancias:** Se calcula la distancia $d(\mathbf{x}_{new}, \mathbf{x}_i)$ entre el nuevo punto y todos los puntos $\mathbf{x}_i$ en el conjunto de entrenamiento.
2. **Identificar vecinos:** Se identifican los K puntos del conjunto de entrenamiento que tienen las distancias más pequeñas a $\mathbf{x}_{new}$. Este conjunto de puntos se denomina $\mathcal{N}_K(\mathbf{x}_{new})$.
3. **Votación por mayoría:** La predicción de la clase para $\mathbf{x}_{new}$, denotada como $\hat{y}_{new}$, se determina por la clase mayoritaria dentro de sus K vecinos:

$$\hat{y}_{new} = \underset{c \in \{\text{clases}\}}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{N}_K(\mathbf{x}_{new})} I(y_i = c)$$

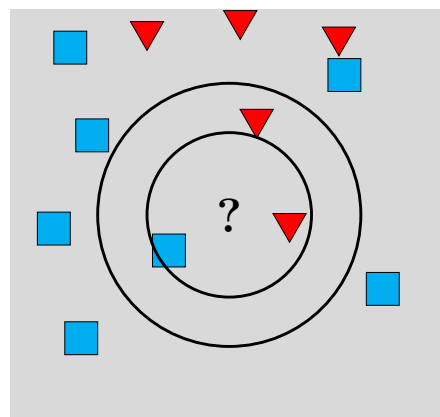


Figura 2.6: La categoría objetivo del punto ? varía con el número de nodos más adyacentes.

donde $I(\cdot)$ es la función indicadora.

Según la explicación anterior podemos darnos cuenta que el algoritmo KNN trae consigo dos decisiones fundamentales: la elección del número de vecinos K y la métrica de distancia, el valor de K nos puede ayudar a controlar el equilibrio entre sesgo y varianza, mientras que la distancia nos ayuda a medir la proximidad entre las nuevas observaciones y los grupos de categorías. Por lo cual asegurar la elección del número de vecinos K y la métrica de distancia d es una decisión crucial, ya que definen por completo el comportamiento del modelo y su capacidad para generalizar a nuevos datos.

La métrica de distancia: Como mencionamos anteriormente el concepto de “proximidad” o “similitud” es la esencia del modelo K Nearest Neighbors, es la forma en que medimos que tan parecidos son las observaciones entre sí y esto se hace haciendo uso la idea de distancia o métrica. Para elegir la métrica adecuada es necesario tomar en cuenta el escalado de las características, por ejemplo, si nuestras variables predictoras se encuentran en distintas escalas, ingresos en miles de millones de pesos, antigüedad en meses o cantidad de productos, la variable que tenga la mayor escala dominará el cálculo de la distancia, ensombreciendo la contribución de las demás. Por ende, es una buena práctica normalizar o estandarizar los datos antes de aplicar KNN.

Las métricas más comunes son:

Definición 2.2.5: Distancia Euclidiana (L2)

Dada la existencia de dos puntos, $\mathbf{p} = (p_1, p_2, \dots, p_n)$ y $\mathbf{q} = (q_1, q_2, \dots, q_n)$, en un espacio n -dimensional, la distancia euclidiana se define como la raíz cuadrada de la suma de las diferencias al cuadrado de sus coordenadas correspondientes.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Esta métrica corresponde a la norma L2 del vector diferencia $\mathbf{q} - \mathbf{p}$.

Para comprender esta métrica podemos pensarlo como medir una línea recta, que es la forma más intuitiva que tenemos de medir cosas en la vida real, suele usarse cuando tenemos datos geoespaciales, cuando el conjunto de datos es denso y tenemos pocas dimensiones y cuando las variables predictoras son de naturaleza similar y continuas, ejemplo pesos, alturas, etc. La única desventaja es que es muy sensible a outliers.

La siguiente distancia es conocida como distancia de Manhattan, distancia del taxista o distancia de la ciudad, a continuación mostramos su definición.

Definición 2.2.6: Distancia de Manhattan (L1)

Para dos puntos $\mathbf{p}$ y $\mathbf{q}$ en un espacio n -dimensional, la distancia de Manhattan es la suma de las diferencias absolutas de sus coordenadas. Es la distancia que se recorrería entre dos puntos si solo se permitieran movimientos a lo largo de los ejes.

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n |q_i - p_i|$$

Esta métrica corresponde a la norma L1 del vector diferencia $\mathbf{q} - \mathbf{p}$.

Esta métrica es conveniente usarla en espacios de alta dimensionalidad, cuando las características son dispares o no tienen correlación aparente, cuando tienes variables muy sesgadas y por lo tanto quieres evitar la sensibilidad a los outliers.

Finalmente tenemos la distancia de Minkowski, la cual es una generalización de las dos anteriores y está definida de la siguiente manera.

Definición 2.2.7: Distancia de Minkowski (Lp)

Es una generalización de las distancias Euclidiana y de Manhattan que introduce un parámetro $p \geq 1$. Para dos puntos $\mathbf{p}$ y $\mathbf{q}$ en un espacio n -dimensional, se define como:

$$d(\mathbf{p}, \mathbf{q}) = \left(\sum_{i=1}^n |q_i - p_i|^p \right)^{1/p}$$

Casos especiales de esta distancia son:

- $p = 1$: Distancia de Manhattan.
- $p = 2$: Distancia Euclidiana.
- $p \rightarrow \infty$: Distancia de Chebyshev.

De igual manera la definición de valor de K en el algoritmo, se debe hacer en el momento de la construcción del modelo, a K se le conoce como hiperparámetro¹, es importante remarcar que no existe un valor de K que se acople a todo tipo de conjunto de datos, sino que el valor es subyacente a las propiedades del conjunto de datos en cuestión. En general una pequeña cantidad de vecinos baja tendrá en consecuencia un bajo sesgo, pero una gran cantidad de vecinos tendrá menor variabilidad pero un mayor sesgo, por lo cual elegir el número de vecinos es una tarea primordial.

¹Suele definirse al hiperparámetro como una variable externa al modelo, es decir, no se obtiene a partir de los datos, sino que debemos definir al momento de implementar el algoritmo.

2.2.1.3. Random Forest

De igual forma que en los modelos anteriores, ahora vamos a explicar el algoritmo Random Forest, expuesto por primera vez por Leo Breiman en 2001, cabe mencionar que este modelo es uno de los más importantes pertenecientes a una clase llamada algoritmo de ensamble. Su contrucción esta basada en la unión de multiples arboles de decisión durante el entrenamiento, esto nos permite emitir una clasificación que es el valor más común (moda) de las clases predichas por los árboles individuales. Por lo cual para entender al cien por ciento el funcionamiento de este modelo primero hay que hablar sobre su componente primario, los arboles de decisión.

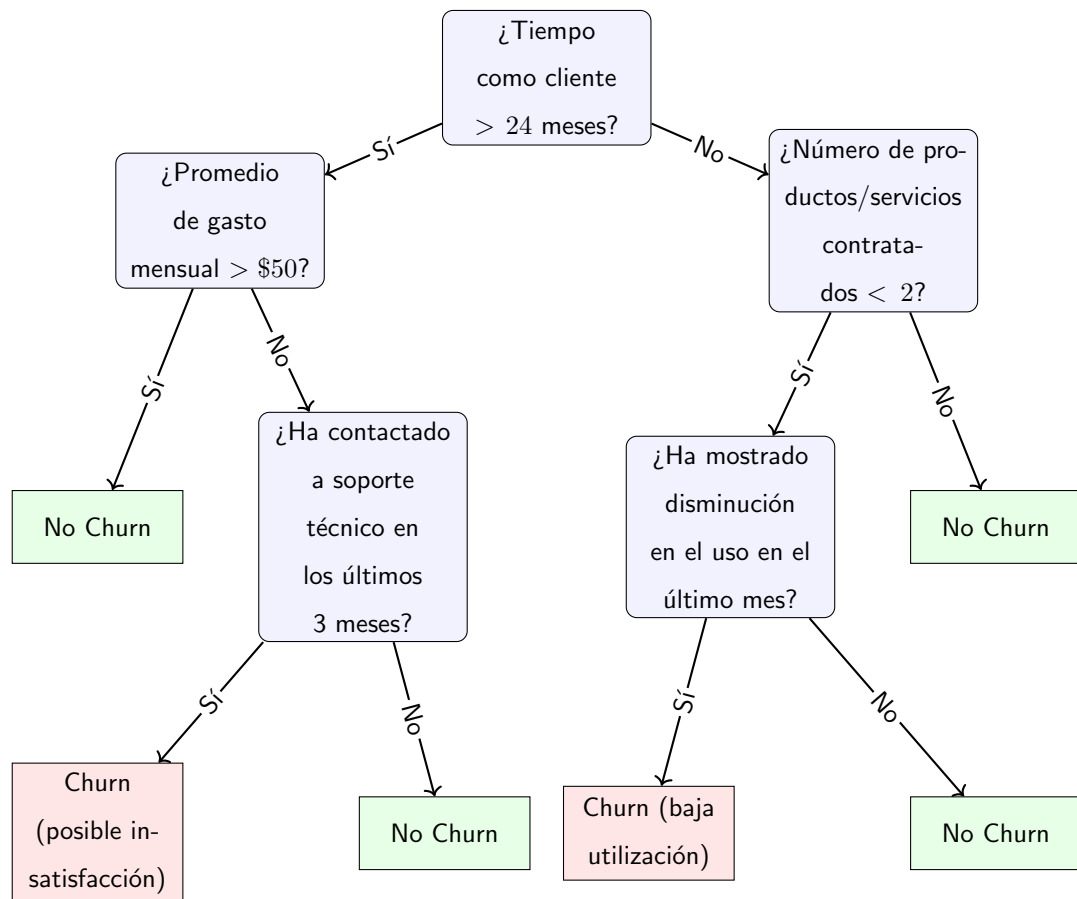


Figura 2.7: Ejemplo de árbol de decisión (con posicionamiento manual) para la predicción de Churn.

Un árbol de decisión en un modelo no paramétrico, este algoritmo nos permite predecir el valor de una variable target, es utilizado tanto en tareas de clasificación y regresión, el modelo aprende ciertas reglas de decisión simples deducidas a partir de las características de los datos. Su estructura es la de un árbol jerárquico, muy

2. MARCO TEÓRICO Y CONCEPTUAL

parecida a la estructura de datos, consta de un nodo raíz, ramas, nodos internos y nodos hojas. Diagramas como 2.7 suele ser muy útiles a la hora de explicar cómo funciona el algoritmo, cada nivel representa una pregunta y las respuestas serán sí o no, de tal forma que eventualmente una respuesta te dará una solución al problema inicial.

Por ejemplo el nodo “¿Tiempo como cliente > 24 meses?”, lo podemos indentificar como nodo raíz, es el inicio de nuestro camino, por otra parte los nodos “¿Promedio de gasto mensual > \$50?” y “¿Número de productos/servicios contratados < 2?” son nodos internos, mientras que los nodos que se encuentran al mismo nivel que estos últimos, son nodos hoja, quiere decir que es el una de las salidas de nuestro algoritmo y finalmente una rama es la ruta que se sigue desde el nodo raíz hasta cualquier otro nodo. El proceso que se lleva a cabo para la construcción de un árbol de decisión, comúnmente se llama partición recursiva, este algoritmo es la medula espinal del modelo de árbol de decisión, además que su funcionamiento es fácil e intuitivo. La premisa es simple, los datos se parten repetidamente apoyándose de los datos de entrenamiento para tratar de conseguir las particiones más homogéneas posibles.

La homogeneidad pese a parecer un concepto trivial, se formaliza a través de la coeficiente de impureza Gini y la entropía, decimos que un nodo es “puro”, es decir con impureza cero, si todas las muestras que contiene pertenecen a la misma clase. Por lo tanto, el algoritmo estima cada posible partición y selecciona aquella que maximiza la reducción de entropía o la división que tenga la menor impureza de Gini ponderada en los nodos hijos, el coeficiente de Gini se define de la siguiente manera.¹

Definición 2.2.8: Coeficiente de Impureza de Gini (Caso Binario)

Sea p la proporción de muestras pertenecientes a una de las dos clases en un nodo. La impureza de Gini (G) para ese nodo se calcula con la siguiente fórmula:

$$G = 2p(1 - p)$$

Alternativamente, si p_1 es la proporción de la clase 1 y p_2 es la proporción de la clase 2 (donde $p_1 + p_2 = 1$), la fórmula general es:

$$G = 1 - (p_1^2 + p_2^2)$$

De igual manera, otro concepto que hay que tener en cuenta es como se lleva a cabo la selección de las mejores particiones posibles, si lo vemos desde cierto punto de vista podemos hablar de un algoritmo codicioso que busca en cada paso, tener la mejor elección de una partición posible, sin importar que más adelante dicha división sea impura o poco óptima, esto se refuerza con la idea de la imposibilidad de tener un árbol globalmente óptimo, es decir, con todas las subparticiones perfectamente homogenizadas, por lo cual este enfoque nos permite darle un giro de tuerca, heurísticamente hablando,

¹De hecho la definición del coeficiente de impureza de Gini se puede extender a un problema de clasificación multiclase de la siguiente manera $G = 1 - \sum_{j=1}^K p_j^2$.

al problema de la creación de un árbol de decisión lo más optimizado posible. Según [Bruce et al. \(2020\)](#) podemos definir el algoritmo de partición recursiva de la siguiente manera.

Definición 2.2.9: Algoritmo de Partición Recursiva Binaria

Dado un conjunto de datos o nodo A con una variable de respuesta Y y un conjunto de M variables predictoras X_j con $j = 1, 2, \dots, M$, el algoritmo busca de manera codiciosa la división óptima que segmenta a A en dos nodos hijos, $A_{izquierda}$ y $A_{derecha}$. Tal que:

1. Para cada variable predictora X_j :
 - a Para cada posible punto de corte s_j de la variable X_j :
 - i Se genera una partición tentativa dividiendo las observaciones en A en dos subconjuntos: $A_{izquierda}(j, s_j) = \{X|X_j < s_j\}$ y $A_{derecha}(j, s_j) = \{X|X_j \geq s_j\}$.
 - ii Se calcula la impureza de los nodos hijos resultantes utilizando una métrica predefinida (ej. Gini o Entropía para clasificación; SSE para regresión).
 - b Se selecciona el punto de corte s_j^* que produce la máxima reducción de impureza para la variable X_j .
2. Se selecciona la variable X_j^* y su punto de corte asociado s_j^* que, entre todas las variables y sus respectivos mejores cortes, ofrezca la máxima reducción de impureza global para el nodo A .
3. El proceso se repite de forma recursiva en los nuevos nodos hijos ($A_{izquierda}$ y $A_{derecha}$) hasta que se satisface un criterio de parada.

Finalmente otra idea que hay que considerar es la detención del crecimiento del árbol o el control de la complejidad de un árbol de decisión, [Bruce et al. \(2020\)](#) menciona que a medida que un árbol de decisión crece, las reglas de división se vuelven más detalladas y este pasa de identificar gradualmente “grandes” reglas, que identifican relaciones reales y confiables en los datos, a “pequeñas” reglas que solamente generan ruido. Entender esto es crucial, ya que en algunos casos permitir el desarrollo desmedido de un árbol de decisión puede acarrear como consecuencia el aprender completamente el conjunto de datos de entrenamiento, lo cual nos llevaría a un problema de sobre ajuste, cosa que queremos evitar preferentemente. Pese a que existen varias formas de podar nuestros árboles de decisión, la practica suele sugerir inclinarse por métodos como la validación cruzada (que más adelante explicaremos) los cuales nos permiten jugar con distintas combinaciones de hiperparametros, en este caso, profundidad maxima de un árbol, el minimo de divisiones posibles, entre muchos otros. Para que así podamos encontrar el mejor modelo posible.

2. MARCO TEÓRICO Y CONCEPTUAL

Para solucionar el problema del sobreajuste, Random Forest emplea una técnica de ensamble llamada Bootstrap Aggregating, o Bagging. La idea central del Bagging es reducir la varianza de un estimador promediando las predicciones de múltiples modelos entrenados en diferentes subconjuntos de los datos.

El algoritmo de Random Forest introduce una capa adicional de aleatoriedad al proceso de Bagging, lo que resulta en una mayor diversificación de los árboles y, en general, en un mejor rendimiento.

Definición 2.2.10: Algoritmo Random Forest

Para un conjunto de entrenamiento con N muestras y M características, se construye un bosque de B árboles de la siguiente manera:

1. Para $b = 1, \dots, B$:
 - a) Se extrae una muestra bootstrap (una muestra aleatoria con reemplazo de tamaño N) del conjunto de entrenamiento.
 - b) Se construye un árbol de decisión sobre esta muestra. En cada nodo del árbol, en lugar de buscar la mejor división entre las M características, se selecciona un **subconjunto aleatorio de $m < M$ características** y se busca la mejor división solo dentro de ese subconjunto.
2. El árbol se crece hasta su máxima profundidad posible sin realizar podas.

La predicción final para una nueva observación se obtiene agregando las predicciones de los B árboles, generalmente mediante una votación por mayoría para clasificación.

Esta doble aleatorización (muestreo de datos y muestreo de características) es la clave del éxito de Random Forest. Asegura que los árboles individuales no estén correlacionados entre sí. Al promediar las predicciones de un gran número de modelos descorrelacionados, el error debido a la varianza se reduce drásticamente, sin aumentar significativamente el sesgo.

Una de las características más valiosas de Random Forest, especialmente para el objetivo explicativo de esta tesis, es su capacidad intrínseca para calcular la importancia de las características (feature importance). Esto se logra midiendo, para cada característica, cuánto disminuye en promedio la impureza (usando el índice de Gini) en todos los árboles del bosque cada vez que esa característica es utilizada para una división. Esto proporciona una métrica cuantitativa y robusta para clasificar qué variables son las más influyentes en la predicción del churn.

2.2.1.4. SVM

2.3. Predicción del churn.

En esta sección abordaremos todos los conceptos concernientes con el churn,

2.3.1. Definición de *churn*

De hecho Gold (2020, p. 36) define el origen de la palabra *churn* en el término *churn rate*, este termino hace referencia a la proporción de clientes que abandonan la empresa en un periodo determinado.

Más en específico el *churn* es un estado u estatus que se le da a un cliente habitual de una empresa o servicio cuando este da por terminada la relación comercial, este concepto ha sido acuñado más recientemente por aquellas empresas que prestan servicios de suscripciones, como pueden ser plataformas de entretenimiento digital, proveedores de servicios de telecomunicaciones (internet, telefonía y/o televisión), etc. Sin embargo el *churn* no es únicamente aplicable a este tipo de compañías.

Ya que solo necesitamos que la empresa en cuestión oferte uno o más productos de los cuales tenga consumidores habituales para poder estudiar el *churn* en toda su cartera de clientes y más en específico poder abordar distintas estrategias para la retención de clientes.

En general el estudiar el comportamiento de los clientes puede ser un reto distinto dependiendo de la empresa e industria en la cual el científico de datos o analista de datos se encuentre, por lo cual es necesario tener pleno conocimiento sobre las reglas de negocio en cuestión. Ya que cada caso de estudio necesitará de enfoques distintos para poder entender de mejor manera el *churn* y cómo combatirlo.

De hecho ese debería ser el propósito principal de estudiar el *churn*, poder proporcionar una radiografía certera sobre el comportamiento de los clientes en una empresa, esto eventualmente permitirá a los líderes dentro de la organización poder tomar decisiones basadas en datos, y es tarea del científico de datos proveer la información lo más clara y precisa posible.

2.3.2. Cálculo del *churn*

Anteriormente expusimos la diversidad de enfoques del *churn*, que hay empresas dedicadas a ofertar servicios de suscripciones o que simplemente tienen una cartera de clientes habituales, pese a todo esto no todas las empresas tienen un indicador de abandono, de hecho en la mayoría de los casos lo primero sea determinar de manera correcta cuando un cliente a abandonado.

Para ellos en nuestro conjunto de datos asignaremos como *churn* a una columna cuyos registro sea true o false, esta fungirá como una bandera de abandono donde el

2. MARCO TEÓRICO Y CONCEPTUAL

valor `true` indica que un cliente abandono y `False` el caso contrario, como construir esta columna sera parte importante de este trabajo.

En el caso de las empresas que ofrecen suscripciones puede no aplicar, ya que saber cuando un cliente ha abandonado la compañía es más sencillo, podemos simplemente asignar el valor de `True` en la columna `churn` a todos aquellos que no cuentan con una suscripción activa y como `False` a los casos contrarios, y como mencionamos anteriormente, hablaremos de como construir el indicador de `churn`, para usar esta forma de determinar el `churn` es necesario contar con una variable que nos indique el tiempo desde la ultima transacción del cliente, y aun que no contemos con este dato podríamos determinarlo fácilmente de la siguiente manera:

```
1 fecha_ref = pd.Timestamp(datetime.now().date())
2 df['Tiempo_ultima_compra'] = df.groupby('bp_id')['Date_Last'].
   transform(lambda x: np.abs((fecha_ref - x.max()).days))
3
```

Donde `'fecha_ref'` es la variable que almacena la fecha del sistema cuando se ejecuta el programa, otra alternativa sería asignar a `'fecha_ref'` un día en específico, ejemplo el fin de mes. Por otra parte necesitamos una columna auxiliar en nuestro dataframe para realizar el calculo final que es el tiempo desde la ultima compra, esta columna es `'Date_Last'`, esta contiene la ultima fecha de compra del cada cliente y en base a esta calcularemos el tiempo transcurrido desde la ultima compra, tal y como se muestra en el código ??.

Una vez determinada la columna `'Tiempo_ultima_compra'` conviene analizar la distribución de los valores para dicha variable, esto nos permite ver el comportamiento general de cada individuo¹, en este caso, podemos visualizar el histograma de la variable en cuestión (figura 2.8) en la cual podemos apreciar que gran parte de los clientes no sobrepasan los 125 días de inactividad de compra, y de hecho podemos notar una linea vertical de color roja que indica el percentil 75² de la variable `'Tiempo_ultima_compra'` lo cual nos indica que los clientes con un tiempo de inactividad demasiado alto son realmente pocos.

¹En el capítulo 2 repasamos el análisis univariado y sus implicaciones teóricas, en esta sección nos enfocaremos unicamente en las aplicaciones.

²Para este ejemplo, decidí tomar el percentil 75, sin embargo la elección de percentil es a criterio del científico de datos y esta fuertemente ligado a las características de los datos con los que estemos trabajando, así que tómese este valor como lo que es, un ejemplo.

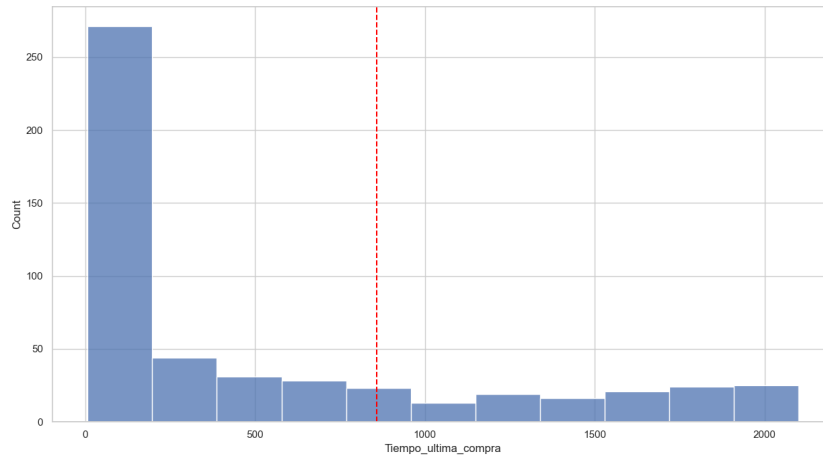


Figura 2.8: Histograma de Tiempo desde la ultima compra

Como sugerencia, conviene que incluyamos una columna en nuestro dataframe que contenga el percentil que deseamos tomar como condición para determinar el abandono, puedes seguir el ejemplo que se muestra en el código 2.1 como ejemplo de como realizar el calculo del k-esimo¹ percentil.

```
1 df['percentile_k'] = np.percentile(df['Tiempo_ultima_compra'], k)
```

Listing 2.1: Ejemplo de como determinar el percentil n

Una vez hecho el análisis del comportamiento de las ultima compra registrada de cada clientes y de haber calculado el percentil es hora de determinar la columna *churn* dentro de nuestro dataframe, por lo cual nos apoyaremos del código 2.2.

```
1 df['Churn'] = np.where(df['Tiempo_ultima_compra'] > df['percentile_k'], True, False)
```

Listing 2.2: Ejemplo de como determinar el churn.

Por ultimo, en la figura 2.9 podemos apreciar la proporción del *churn* en los clientes a analizar, y el resultado de la codificación anterior, en la gráfica de pie podemos notar que existe cierto desbalance en la proporción de clientes abandonadores y no abandonadores, recordemos que a posteriori esto puede ser un problema en la implementación de modelos de clasificación binaria, lo cual fue abordado en el capítulo 2.

¹Donde k es el valor del percentil a determinar.

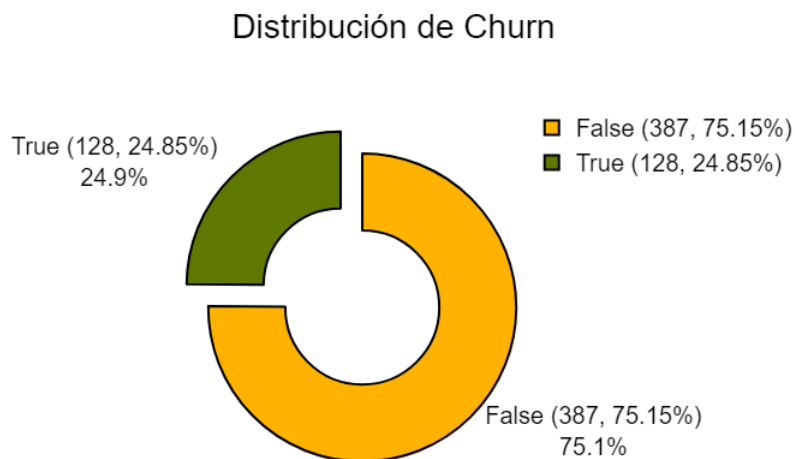


Figura 2.9: Gráfica de tarta para el abandono de clientes.

En estas ultimas secciones he abordado a profundidad el concepto de *churn*, el como analizar la actividad de compra de los clientes nos ayuda a comprender a como determinar el *churn*, por lo cual en la siguiente sección realizaremos un tipo de análisis más enfocado al comportamiento histórico de los clientes y ver cual ha sido el *churn* y retention rate.

2.3.3. Metricas y causalidad del *churn*

Pese a que existen numerosas estrategias que podrían ayudar a disminuir la tasa de abandono y por lo tanto combatirlo, cabe resaltar que no nos enfocaremos en explicar dichas estrategias, más bien, nos enfocaremos en como aprovechar las razones subyacentes al *churn* para poder definir buenas métricas ¹ que ayuden a evaluar el comportamiento de los clientes, y así poder usar estas mismas métricas en una eventual predicción del *churn*, el cual es propósito final de este trabajo.

Una definición de métrica para clientes puede ser la provista por Gold (2020, p. 51) “*cualquier medición que realice sobre todos los clientes individualmente*”. Por lo cual es crucial encontrar las mejores metricas para monitorear a los clientes, de hecho Gold (2020) menciona las siguientes características importantes que debe tener una buena métrica; debe ser fácil de entender para la empresa, deben estar asociada con el

¹En este caso cabe aclarar que otros sinónimos con los nos referiremos a las metricas son características, dimensiones o variables predicadoras.

churn y la retención, segmenta a los clientes de manera que facilita las intervenciones dirigidas a disminuir el *churn rate* y sobre todo es útil para múltiples áreas de la empresa (marketing, soporte, etc.)

En el capítulo anterior discutimos sobre la importancia de seleccionar correctamente las características para implementar *modelos de clasificación binaria*, mencionamos diversas técnicas de reducción de dimensiones basadas en criterios estadísticos, sin embargo, hay que considerar otras pautas a la hora de escoger dichas dimensiones si lo que deseamos es predecir el *churn*, estos juicios alternativos obedecen más a las reglas del negocio o empresa. Claro siempre hay que analizar la relación que pudieran tener con el *churn*.

2.3.4. Análisis de Cohortes y la tasa de abandono

Analizar el comportamiento del abandono de clientes a lo largo del tiempo nos puede ayudar a encontrar conductas que a simple vista suelen pasar desapercibidas. Por lo tanto cabe aclarar el significado de cohorte, (Gold, 2020) lo define como un grupo de individuos que son similares, y de hecho lo que nos interesa en este trabajo es estudiar esa similitud a lo largo del tiempo para poder encontrar patrones en el comportamiento del abandono de clientes.

Por lo cual los grupos que definiremos serán periodos de tiempo, agruparemos aquellos clientes que tuvieron actividad durante dichos periodos, en particular para este trabajo definiremos los cohortes como meses, pero bien podrían ser semanas, años, etc. El objetivo de identificar estos cohortes es tener mayor claridad de la actividad mensual de los clientes y poder hacernos las siguiente preguntas. ¿Cuántos clientes activos tiene una empresa por mes? ¿Cuántos clientes nuevos han adquirido? y ¿Cuántos clientes han perdido?

Podemos estudiar el comportamiento de estos cohortes usando diferentes metricas, ya sea por el numero de suscripciones a un servicio, volumen de compra, valor de compra o numero de clientes activos, este ultimo es la que usaremos en este trabajo, ya que esto nos permitirá relacionarlo con un concepto previamente visto, el *churn rate*.¹

Hacer este análisis en python es tarea sencilla, primero es necesario tener la fecha de compra o fecha de facturación contenida en nuestro datafra, cabe aclarar que este ejercicio podría hacerse antes de determinar la variable de abandono, esto para evitar tener la correcta dimensión del dataframe con el que deseamos trabajar, ya que el nivel de detalle que deseamos tener para realizar este análisis es por fecha de compra, consumo, producto y clientes o usuarios. Aclarado lo anterior, podemos comenzar a revisar los pasos a seguir para hacer este análisis:

Duda: ¿Sera mejor mostrar esto como pseudocódigo?

¹Antes de continuar tenemos que aclarar que hay distintos tipos de enfoques para agrupar los clientes, estos enfoques son inherentes al negocio o empresa en el cual se quiera implementar, además esta fuertemente ligado a la información que tengamos disponible, por lo cual hay que tener en cuenta estas limitantes antes de llevar acabo el análisis de cohortes.

2. MARCO TEÓRICO Y CONCEPTUAL

1. Ordenar las observaciones de nuestro dataframe por fecha y id de usuario.¹
2. Crear una variable que almacene el desplazamiento de cada cohorte en meses posteriores, esta variable nos ayuda a identificar los periodos de desplazamiento posterior a la adquisición de los clientes.
3. Determinar el mes de adquisición del cohorte de clientes.
4. Contar los clientes activos mensuales de cada cohorte y sus desplazamientos posteriores.
5. Del conteo final hay que tomar los clientes iniciales de cada cohorte y dividir todos los conteos por el número de clientes iniciales.

A continuación mostraremos la implementación de dicho análisis, el primero de los pasos esta ilustrado en el código 2.3

```
1 df.sort_values(by=['Fecha', 'bp_id'], inplace=True)
```

Listing 2.3: código del paso 1 del analisis de cohorte

Y es que como bien lo indican las instrucciones simplemente tenemos que ordenar nuestro dataframe haciendo uso de la API de pandas.

Ordenar los datos de esta manera nos asegura que los cohortes que creemos tengan coherencia cronológicamente, esto porque en el paso 2 y 3 para determinar el mes de adquisición del cohorte y los meses de desplazamiento nos basamos en los meses de actividad de los clientes, en el código 2.4 creamos la columna *CohortMonth* la cual corresponde al mes de adquisición del cohorte, mientras que *InvoiceMonth* es el mes de compra cada cliente.

```
1 def get_month(x): return dt.datetime(x.year, x.month, 1)
2
3 df['InvoiceMonth'] = df['Fecha'].apply(get_month)
4 grouping = df.groupby('bp_id')['InvoiceMonth']
5 df['CohortMonth'] = grouping.transform('min')
```

Listing 2.4: código del paso 2 del analisis de cohorte

De forma auxiliar y previo obtener el resultado final, necesitamos crear una función que extraiga los valores enteros de los campos *CohortMonth* y *InvoiceMonth*, en el código 2.5 mostramos la definición de dicho método.

```
1 def get_date_int(df, column):
2     year = df[column].dt.year
```

¹Es necesario que la variable que contenga la fecha sea del tipo de datetime para poder obtener una salida correcta en este paso y en los posteriores.

```

3 month = df[column].dt.month
4 day = df[column].dt.day
5 return year, month, day

```

Listing 2.5: código para el método `get_date_int`

Análogamente para mejorar la forma en que vemos la información obtenida en el paso 3 nos apoyamos de la función `get_date_int`, el código 2.6 nos ayudará a crear una columna llamada `CohortIndex`, esta columna como su nombre es un conjunto de índices cuyo fin es representar los cohortes, osea nos permite tener una mejor visualización del desplazamiento de cada grupo de clientes.

```

1 invoice_year, invoice_month, _ = get_date_int(df, 'InvoiceMonth')
2 cohort_year, cohort_month, _ = get_date_int(df, 'CohortMonth')
3 years_diff = invoice_year - cohort_year
4 months_diff = invoice_month - cohort_month
5 df['CohortIndex'] = years_diff * 12 + months_diff + 1

```

Listing 2.6: código para la obtención de la columna `CohortIndex`

Por otra parte, para obtener el paso 4 debemos hacer uso de una pivot table agrupar los datos por `CohortMonth` y `CohortIndex`, así como agregar para obtener el resultado final, en el código 2.7 podemos observar como llevar acabo esta implementación. La utilidad de usar una tabla pivote para mostrar el resultado es porque gracias a esta forma podremos visualizar de mejor manera el desplazamiento desde el momento de adquisición para cada cohorte hasta la ultima actividad.

```

1 grouping = df.groupby(['CohortMonth', 'CohortIndex'])
2 cohort_data = grouping['pd_id'].apply(pd.Series.nunique)
3 cohort_data = cohort_data.reset_index()
4 cohort_counts = cohort_data.pivot(index='CohortMonth',
5 columns='CohortIndex',
6 values='pd_id')

```

Listing 2.7: código para la obtención de la columna `CohortIndex`

Finalmente para poder obtener la matriz de tasa de retención tenemos que llevar acabo el paso 5, la implementación la mostraremos en fragmento de código 2.7 se crea la variable `cohort_sizes` que toma la primera columna, esta representa el número de clientes iniciales de cada cohorte, una vez hecho esto crearemos la tabla pivote final llamada `retention`, la misma es resultado de dividir toda la pivot `cohort_counts` por `cohort_sizes`, tal y como se muestra en la ecuación 3.1, de hecho ya podemos explicar el porque la ecuación del `churn rate` considera el numero de clientes por periodos de tiempo, y es que esta es la mejor forma de analizar la tasa de retención.

2. MARCO TEÓRICO Y CONCEPTUAL

```
1  \# Dependencias
2  import seaborn as sns
3  import matplotlib.pyplot as plt
4  \# Calulo de la matriz de retencion
5  cohort_sizes = cohort_counts.iloc[:,0]
6  retention = cohort_counts.divide(cohort_sizes, axis=0)
7  retention.round(3) * 100
8  \# Visualizacion de matriz de retencion.
9  plt.figure(figsize=(35, 30))
10 plt.title('Retention rates')
11 sns.heatmap(data = retention,
12 yticklabels=df['CohortMonth'].sort_values().apply(lambda x: x.strftime
13             ("%Y %b, %d")).unique(),
14 annot=True,
15 linewidths=.2,
16 fmt='.0%',
17 cmap='coolwarm',
18 vmin=0,
19 vmax=1)
20 plt.show()
```

Listing 2.8: código para la obtención de la columna *CohortIndex*

La gráfica 2.10 nos muestra el resultado final de las anteriores codificaciones, en los registros correspondientes a la primera columna podemos observar que la tasa de retención es siempre del 100 % esto se debe a que son los clientes que tienen su primer actividad de compra dentro de ese cohorte, de tal forma que los registros de las columnas posteriores representan la actividad de los clientes después de su adquisición, por ejemplo, el cohorte enero de 2022 en el segundo mes¹ (febrero de 2022) podemos observar una retención del 76 % de los clientes iniciales y para el ultimo indice podemos notar que se mantuvieron 64 % de los clientes que iniciaron en enero del 2022, esto quiere decir que en “Fecha final” se perdió el 36 % de los clientes ingresados en enero

¹Ya que cada numero es una representación simplificada de cada cohorte, es importante tomar en cuenta que dicho numero se toma apartir del cohorte inicial, por ejemplo, para el cohorte enero de 2022 el numero 1 es igual al mismo mes, en cambio el numero 2 significa febrero de 2022, de forma similar el numero 3 es equivalente a marzo del 2022 y así sucesivamente hasta llegar al ultimo mes, por consecuencia ultimo mes solo tendrá datos hasta el indice 1, el penúltimo contiene información hasta el indice 2, etc.

de 2022, este dicho grupo de clientes tiene un comportamiento de compra habitual y guardan cierta fidelidad.

Similarmente, los clientes adquiridos en agosto de 2022 tienen un comportamiento similar hasta 6 meses después de la adquisición, sin embargo es a partir del mes 7 que este comportamiento baja al 40 %. El general la tasa de retención de clientes adquiridos en cada cohorte es bajo , ya que para el mes más reciente de cada cohorte podemos notar una baja tasa de retención, lo cual indica que la empresa ha perdido más clientes de los que ha ganado en los últimos meses.

2.4. Inteligencia artificial explicable (XAI)

2.4.1. SHAP

2.4.2. LIME

2.4.3. Importancia de características

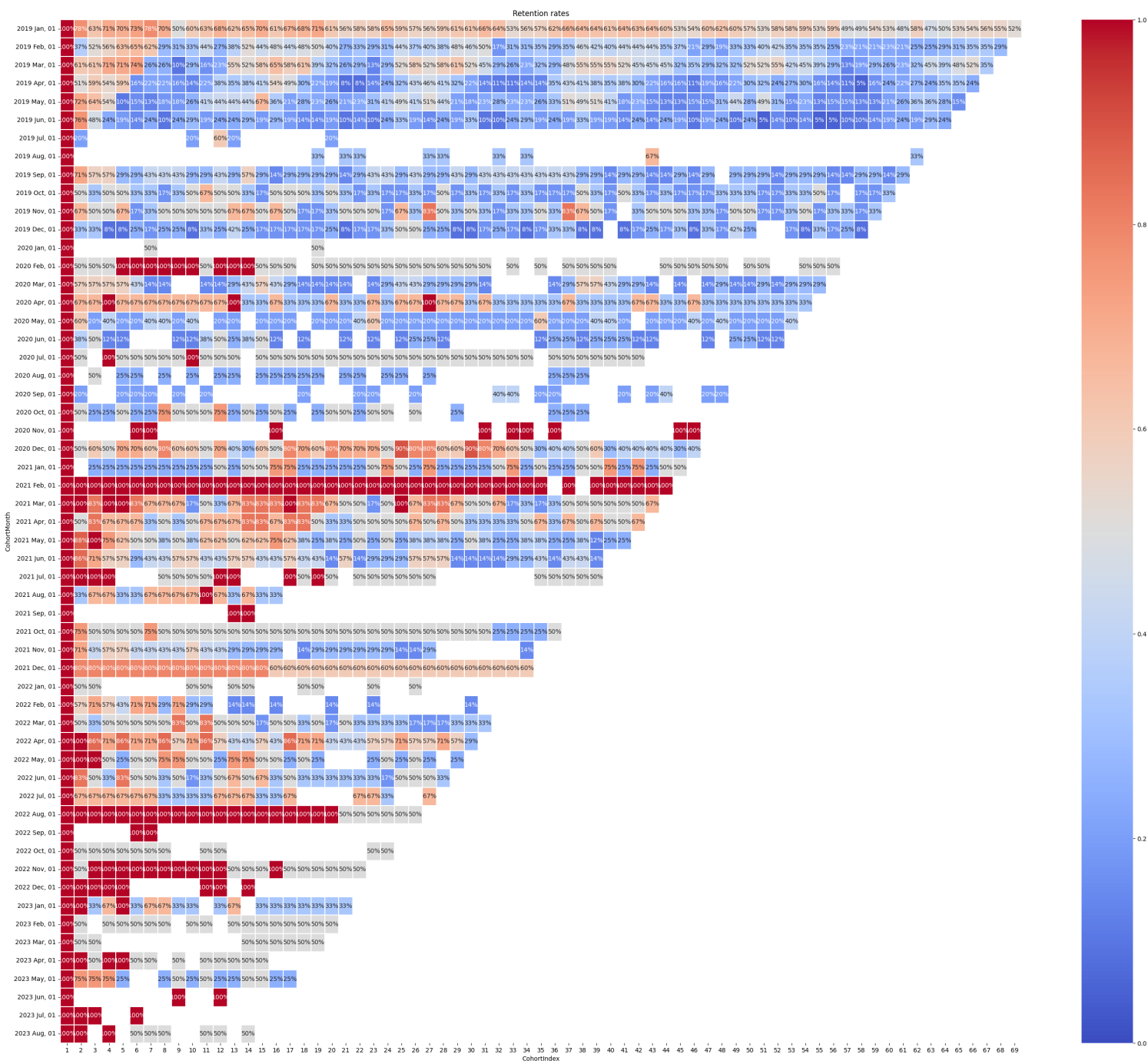


Figura 2.10: Matriz de Cohortes.

Metodología de análisis y preparación de los datos

3.1. Descripción del conjunto de datos

Una parte crucial en toda implementación de Machine Learning es la calidad de los datos, y es algo que cuando se hacen implementaciones sobre información real cobra mucha relevancia, ya que en términos generales es deber del científico de datos a cargo, asegurarse que dicha información sea de calidad.

Ya sea asegurarse del cumplimiento de los supuestos que cada modelo de machine learning demanda ó bien, asegurarse que la calidad de la información sea la adecuada para poder obtener las mejores estimaciones y evitar problemas de Under-fitting o Over-fitting.

En la mayoría de las implementaciones de Machine Learning necesitamos recolectar la información desde alguna fuente de origen, ya sea una base de datos, data lake o cual sea la fuente de la que provengan nuestros datos, por lo cual sera de vital importancia tener total conocimiento acerca del sistema de origen de la información. En consecuencia parte del pipeline que se va a preparar comprende la extracción de la información y en algunos casos preparar un primer procesamiento que nos asegure trabajar con el nivel de especificación que deseamos.

Para el caso particular presentado en este trabajo, es necesario asegurarnos que el nivel máximo de detalle en nuestros datos sean clientes, osea que cada campo y registro debe hacer referencia a información de los clientes, como su historial de compras, año de registro, etc. Y por lo tanto un primer paso sería validar que estemos trabajando con registros únicos.

En consecuencia es importante conocer todos los detalles posibles de los datos con los que vamos a trabajar, tanto en lo concerniente al negocio o a las propiedades inherentes de los datos, esto sera mejor abordado en el siguiente apartado.

3.2. Preprocesamiento

3.2.1. Limpieza y tratamiento de valores faltantes y outliers

3.2.2. Transformaciones y codificación

3.2.3. Selección y reducción de características (PCA)

3.3. Construcción de la variable de abandono (churn)

En este capítulo abordaremos el concepto de abandono de clientes, al ser este uno de los fenómenos que deseamos analizar y predecir, es de suma importancia comprender el concepto de abandono, esto para tener completo entendimiento sobre el comportamiento de los clientes y el fuerte impacto que puede tener el conocer el abandono dentro de una organización. En adelante nos referiremos al abandono como *churn* y a la tasa de abandono como *churn rate*.

La forma que usaremos para calcular la tasa de abandono en este trabajo sera la dada por 3.1, como podemos apreciar esta expresión considera cierto enfoque temporal, más adelante abordaremos el porque.

$$\text{Churn Rate} = \frac{\text{Número de clientes en el ultimo periodo}}{\text{Total de clientes}} - 1 \quad (3.1)$$

De hecho si tomamos la razon entre el numero de clientes en el ultimo periodo o en el actual y el total de clientes tendremos la tasa de retención de clientes. Entendiendo que la tasa de retención es el complemento de la tasa de abandono.

3.4. Implementación de modelos de ML para predecir el abandono

Una vez entendido el concepto de retención y abandono de clientes y las tasas y el análisis de cohorte, comenzaremos por mostrar la implementación de algoritmos de Machine Learning que nos permitan realizar predicciones del abandono. Recordemos que definimos la columna churn (abandono) como una variable de tipo binaria, donde true indica que un cliente abandono y false el caso contrario, en este caso nuestra variable objetivo sera la columna churn.

En esta sección nos apoyaremos de las herramientas vistas en la sección 2 y más en específico llevaremos acabo la implementación de modelos de clasificación binaria para poder predecir las etiquetas de la variable churn, por lo cual no profundizaremos mucho en la teoría detrás, sino nos enfocaremos unicamente en los resultados obtenidos y la interpretación de los mismos.

Para este trabajo compararemos los resultados de distintos modelos de clasificación binaria, tomando en cuenta las metricas de evaluación como precisión, exactitud, etc, haciendo uso tanto de herramientas visuales y resúmenes estadísticos veremos las desventajas y ventajas de cada modelo, esto eventualmente nos conducirá a determinar el modelo que mejor realice la clasificación.

Antes de empezar con el análisis comparativo, es conveniente aclarar que los modelos que en adelante analizaremos ya fueron optimizados y entrenados en el capitulo anterior, por lo cual en esta sección tomaremos como punto de partida lo visto en el capitulo anterior.

3.4.1. Evaluación del modelo.

Previo a la profundización en cada modelo y sus resultados veremos como evaluar el rendimiento de cada modelo, en esta sección utilizaremos como ejemplo el modelo multi-layer perceptron classifier, por lo cual lo visto de aquí en adelante es fácilmente reproducible a cualquier otro modelo. Partiremos de las predicciones con el conjunto de datos de validación, así nos apoyaremos de la matriz de confusión y curva roc para evaluar nuestros modelos de forma visual y numérica.

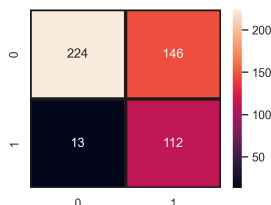


Figura 3.1: Matriz de confusión para el modelo de multi-layer perceptron classifier.

3. METODOLOGÍA DE ANÁLISIS Y PREPARACIÓN DE LOS DATOS

En la gráfica 3.1 podemos apreciar el numero de aciertos (verdaderos positivos y negativos) y el numero de errores (falsos positivos y negativos) cometidos por el modelo. La representación gráfica anteriormente mencionada es la matriz de confusión asociada al modelo de multi-layer perceptron classifier, de hecho esta gráfica nos permite encontrar la mayoría de las metricas para evaluar nuestro modelo. Por ejemplo, la ecuación 3.2¹ nos proporciona la *accuracy* o exactitud, esta métrica nos indica el porcentaje de acierto que tuvo el modelo. Python nos facilita el calculo de esta métrica, el código 3.1 muestra un ejemplo de como calcular la accuracy haciendo uso del sklearn.

Aquí pienso colocar el análisis de la matriz de este modelo, decidí no añadirlo por que puede cambiar.

$$\text{accuracy} = \frac{TP + TN}{(TP + TN + FP + FN)} \quad (3.2)$$

Sin embargo Peterson (2023) menciona que la accuracy usualmente no es una buena métrica para evaluar modelos cuando tenemos clases desbalanceadas en nuestra variable objetivo, lo cual es nuestro caso, ya que como mencionamos anteriormente la variable *churn* tiene más registros marcados como *false*, aun que en el capitulo anterior explicamos la forma en que lidiamos con el desbalance de las clases, explicaremos por que no siempre es conveniente optar por la exactitud, esto porque puede encubrir un incorrecto funcionamiento del modelo, por ejemplo, la matriz 3.1 podemos notar que el total de las observaciones que inicialmente fueron clasificadas como false son mayores a las clasificados como true, el problema es que aunque el porcentaje de aciertos en la clase con dominante es medianamente aceptable, siento mayor a 90 %, la clase minoritaria muestra un menor rendimiento, **insertar porcentaje** esta cifra indica el mal rendimiento para los registros clasificados como true.

Y es que aun que la clase true sea minoritaria, nos interesa tener la mayor cantidad de predicciones correctas para que en un futuro poder predecir correctamente los clientes que están abandonaran la compañía. Por lo tanto recomendamos optar por otras metricas para tener una visión mas completa del rendimiento de nuestro modelo.

```
1 from sklearn.metrics import accuracy_score, confusion_matrix,
   classification_report, precision_score, recall_score, f1_score
2 print("Accuracy: {}".format(accuracy_score(y_true=valid['y'], y_pred=
   valid['y^_MLP'])))
3 print("Precision: {}".format(precision_score(y_true=valid['y'], y_pred=
   valid['y^_MLP'])))
4 print("Recall: {}".format(recall_score(y_true=valid['y'], y_pred=valid[
   'y^_MLP'])))
```

¹Las variables TP, TN, FP y FN son el numero de casos true positives, true negatives, false positives y false negatives respectivamente.

```
5 print("F1: {}".format(f1_score(y_true=valid['y'], y_pred=valid['y^_MLP'])))
```

Listing 3.1: código para calculo de metricas

Algunas de las metricas a las que podemos recurrir son precision (precisión), recall (sensibilidad o recuperación) y F_β . Aun que existe muchas otras metricas, nosotros nos enfocaremos solo en estas. La precisión (ecuación 3.3) es el número de aciertos, predicciones correctas, entre el total de casos positivos captados por el modelo, osea del total de casos clasificados como true, cuales son realmente true, mientras que la sensibilidad (ecuación 3.4) es el numero de predicciones correctas, entre el total de casos positivos, es decir es la proporción de casos clasificados como true.

$$\text{precision} = \frac{TP}{(TP + FP)} \quad (3.3)$$

$$\text{recall} = \frac{TP}{(TP + FN)} \quad (3.4)$$

Hay distintas ventajas de usar una u otra métrica para evaluar nuestro modelo, si nosotros decidimos inclinarnos por la recall, estaríamos dando mayor importancia a reducir la cantidad de errores cometidos por el modelo, esto nos indica que esta métrica no nos aporta información acerca de los falsos positivos, y lo mismo pasa con la precision, ya que al querer reducir el numero de falsos positivos estaríamos ignorando el rendimiento del modelo frente los falsos negativos, por lo cual escoger una u otra es decisión del científico de datos a cargo, en el código 3.1 nos muestra como calcular estas dos metricas con ayuda de Python.

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot (\text{precision} + \text{recall})} \quad (3.5)$$

La métrica que nos permite encontrar un equilibrio entre el numero de FP y FN es el F-score, esta métrica combina tanto la precision y recall en un solo valor, la ecuación 3.5 nos permite calcular dicho valor. El parámetro de β nos da la posibilidad de otorgarle un nivel de importancia a la precision o recall. De hecho si el valor de β es igual a cero el F-score le da mayor importancia a la precision, mientras que cuando β es igual a 0.5 el F - score le da mayor peso a la recall, por ultimo si β es igual a 1, obtenemos el F_1 -score (ecuación 3.6) le damos igual importancia a ambas metricas, de esta forma podemos evaluar tanto el número de falsos positivos y negativos.

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{(\text{precision} + \text{recall})} \quad (3.6)$$

De la comparación entre precision y recall, surge otra forma de evaluar modelos de clasificación binaria, esta métrica es la curva roc (Receiver Operating Characteristic), **Géron** (2017, p. 91) nos dice que *la curva ROC representa gráficamente la tasa de*

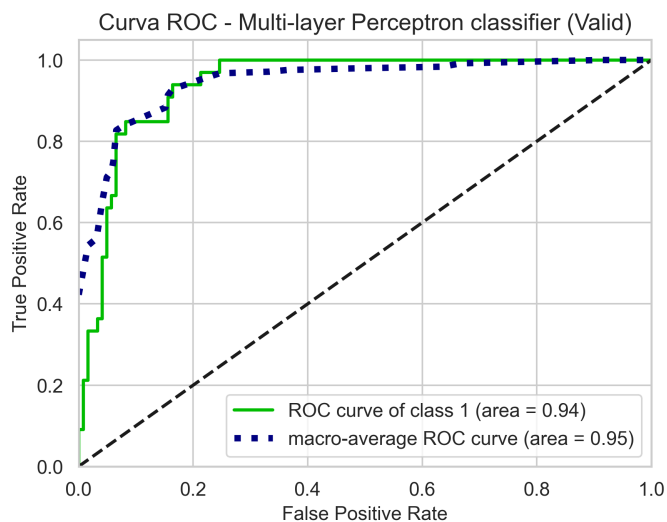


Figura 3.2: Curva ROC.

verdaderos positivos frente a la tasa de falsos positivos. La FPR es la proporción de casos negativos que se clasifican incorrectamente como positivos.

En la gráfica 3.2 podemos notar que el eje de ordenadas está etiquetado como True Positive Rate (TPR), mientras que el eje de abscisas está etiquetado como False Positive Rate (FPR), donde las TPR puede ser también interpretado como la recall mientras que FPR suele interpretarse como el complemento de la especificidad ($1 - \text{especificidad}$), (Géron, 2017) menciona que un buen clasificador se mantiene lo más posible alejado de la línea vertical punteada, dicha línea diagonal sirve más como una referencia y es también llamada línea de no-discriminación.

Además de la curva roc, el área bajo la curva roc (comúnmente llamada AUC) es otra muy buena opción para medir el rendimiento de modelos de clasificación binaria, ya que un valor cercano a 1 de la AUC nos indicará que contamos con un buen clasificador, este buen rendimiento se considera respecto a la especificidad y recall, es decir, estas dos métricas nos indican que nuestro modelo en cuestión dará una mejor calificación a la probabilidad de un registro positivo seleccionado al azar que a un registro negativo seleccionado de forma aleatoria.

De hecho este último punto deja en evidencia que existe cierto inconveniente al evaluar nuestros modelos con la auc solamente, ya que la curva roc puede encubrir desbalances entre clases de la variable objetivo, Chugh (2023) plantea que la curva roc tiende a pasar por alto el desequilibrio entre clases, ya que la tasa de FP es una métrica que solo contempla a la clase negativa, lo que significa que se espera que el cambio en FP sea proporcional al cambio en FP+TN (todos los casos negativos), esto puede derivar en tener poca sensibilidad a cambios en la distribución de las clases de la variable objetivo.

Partiendo de este último punto y recordando que en nuestro caso la información que

estamos analizando tiene cierto desequilibrio entre clases de la variable a predecir, sin embargo no todo esta perdido, ya que existe otra métrica que nos proporciona mayor información que la auc y que no pasa por alto el desequilibrio en entre clases, esta métrica es la curva precision-recall, en palabras de [Chugh \(2023\)](#) *una curva ROC es similar a la curva PR (Precision Recall), pero traza la tasa de verdaderos positivos (TPR) frente a la tasa de falsos positivos (FPR) para diferentes umbrales... La PR es comparativamente más informativa. Esto se debe a que la curva P-R proporciona información más significativa sobre la clase de interés en comparación con la curva ROC.*

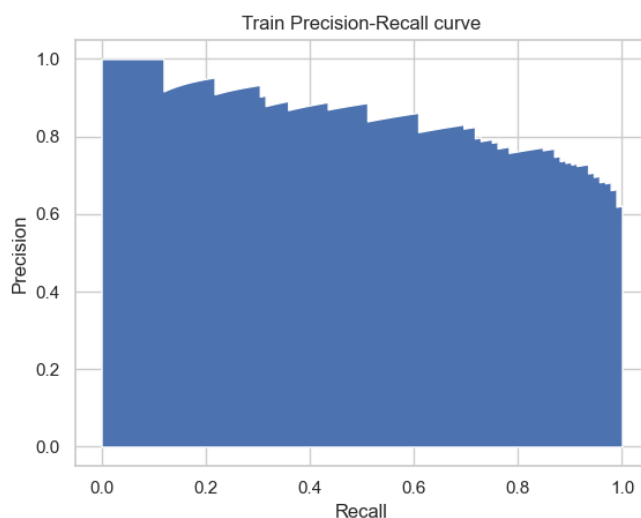


Figura 3.3: Curva PR.

Por lo tanto la curva PR nos ayudara a evaluar de mejor manera nuestros datos dando mayor importancia a los registros positivos pero sin dejar completamente de lado los negativos, encontrando así un equilibrio no arbitrario entre uno y otro. Esto porque uno de los grandes inconvenientes de elegir la auc sobre la PR es que en nuestro caso al tener mayor proporción de registros clasificados negativamente (osea como no abandono) hace que la tasa de FP sea demasiado pequeña, en comparación de la PR que al considerar la a la precision y por ende el rendimiento de nuestro modelo en los registros positivos.

3.4.2. Comparación del rendimiento entre modelos.

Analizaremos el rendimiento de nuestros modelos en el conjunto de entrenamiento y de validación haciendo uso de las metricas antes mencionadas. Por ejemplo, en la matriz de confusión del modelo de regresión logística para el conjunto de entrenamiento ([3.4 izquierda](#)) podemos notar signos de un mal desempeño a la hora de predecir

3. METODOLOGÍA DE ANÁLISIS Y PREPARACIÓN DE LOS DATOS

correctamente los registros positivos (abandonos) con respecto de los casos negativos (no abandonos), esto se hace más evidente en el caso de los falsos positivos, cuya presencia es mayor que los verdaderos positivos, lo cual se traduce inmediatamente en una precisión baja. Sin embargo en el conjunto de validación (derecha 3.4) podemos notar que este comportamiento persiste.

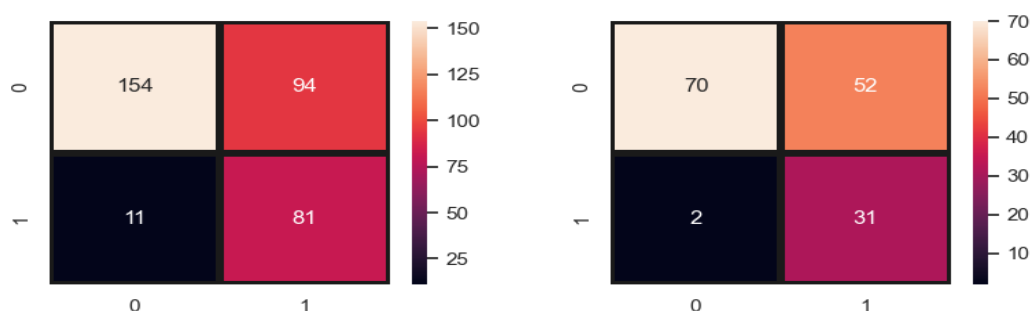


Figura 3.4: Matriz de confusión para el modelo de regresión logística del conjunto de entrenamiento y validación

De hecho nos podemos auxiliar de la gráfica de precisión vs recall (3.5) trazada para el modelo de regresión lineal, en este caso claramente podemos notar que nuestro modelo no está teniendo un buen desempeño, esto se puede hacer más evidente si analizamos la accuracy, cuyo valor es demasiado bajo. En la gráfica 3.6 podemos ver el desempeño de cada una de las métricas antes mencionadas en nuestro modelo, vemos que tanto la accuracy y el F1 score tienen un mal desempeño, esta gráfica nos confirma el mal desempeño de este modelo tanto en el conjunto de entrenamiento y validación.

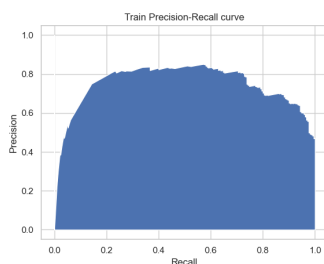


Figura 3.5: Curva PR de Regresión Logística

Por último inspeccionando la curva roc del modelo de regresión logística en la gráfica 3.13, podemos notar los vicios antes mencionados en nuestro modelo, dado que el “ideal” de un modelo es tener una area (auc) lo más cercana a uno, en la curva roc del conjunto de entrenamiento y validación del modelo de regresión logística podemos

notar que la $AUC_{train} = 0.86$ y $AUC_{valid} = 0.85$, esto aunado a que la curva roc es bastante cercana a la linea de no-discriminación, por lo cual podemos considerar a este modelo como un mal clasificador, dado que no cumple con el rendimiento esperado, por las razones antes mencionadas, lo cual eventualmente causaría conflictos, pues hay mayor posibilidad de clasificar de forma equivocada a un cliente como abandonador.

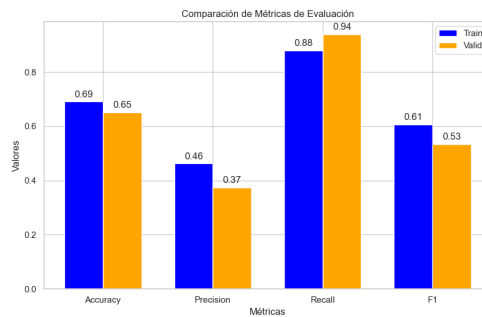


Figura 3.6: Metricas de evaluación del modelo de regresión logística para el conjunto de entrenamiento y validación

Por otra parte, el modelo KNN nos ofrece características más interesante, de inicio la matriz de confusión (3.7) tanto del conjunto de entrenamiento y validación a diferencia del modelo anterior podemos ver un menor numero de casos falsos positivos y por ende un mejor rendimiento a la hora de clasificar los casos verdaderamente negativos y positivos, si analizamos damos una rápida inspección a las metricas de evaluación del modelo podemos notar un mejor rendimiento en la presicion y por ende tenemos un mejor F1 score.

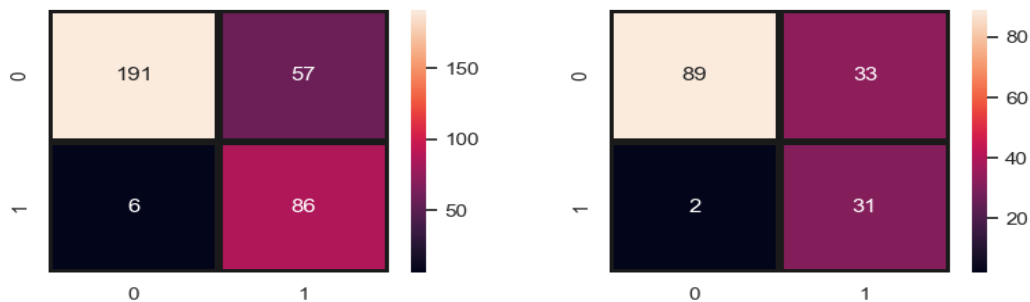


Figura 3.7: Matriz de confusión para el modelo KNN del conjunto de entrenamiento y validación

Esto ultimo lo podemos confirmar con la gráfica 3.8 en donde podemos ver una

3. METODOLOGÍA DE ANÁLISIS Y PREPARACIÓN DE LOS DATOS

mejora significativa en el rendimiento del modelo, teniendo una mejor accuracy e F1 score, sin embargo el modelo presenta un inconveniente que podemos apreciar. El rendimiento del modelo decrece en el conjunto de validación, esto con respecto del conjunto de entrenamiento, esto nos indica que este modelo en específico no generaliza del todo bien las predicciones, de hecho esto se hace evidente en las variaciones tan marcadas en las métricas que muestra el modelo, por ejemplo podemos ver que tan solo la accuracy del conjunto de pruebas y validación varía por 0.05, y que el F1 score varía 0.11, lo cual es una gran diferencia en el rendimiento de nuestro modelo, esto nos da indicios de que este modelo podría volverse obsoleto en el corto o mediano plazo, lo cual podría repercutir en futuros errores de clasificación.

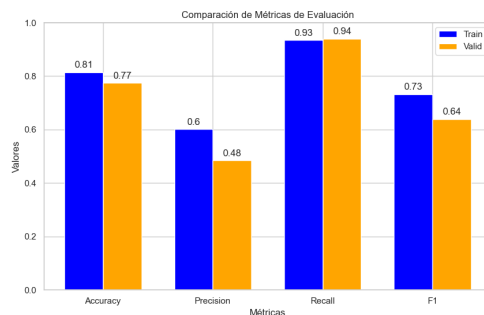


Figura 3.8: Métricas de evaluación del modelo de KNN para el conjunto de entrenamiento y validación

De igual forma la curva roc del modelo KNN que podemos apreciar en las gráficas 3.13 vemos este decaimiento en el rendimiento en el conjunto de validación, notemos como la $AUC_{train} = 0.96$ baja $AUC_{valid} = 0.90$ lo cual confirma que este modelo no es confiable y por tanto lo descartaremos como modelo final para clasificar clientes.

Ahora vamos a analizar los resultados del modelo support vector machine (svm), de forma análoga inspeccionaremos brevemente las matrices de confusión del modelo, podemos notar en primera instancia (gráfica 3.9) que el comportamiento del clasificador en el conjunto de entrenamiento y validación es similar al comportamiento mostrado por los dos modelos anteriores en dichos conjuntos de datos.

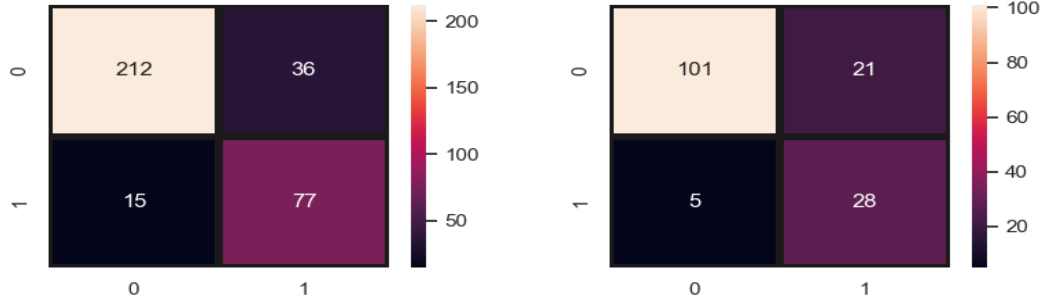


Figura 3.9: Matriz de confusión para el modelo de soporte vector machine del conjunto de entrenamiento y validación

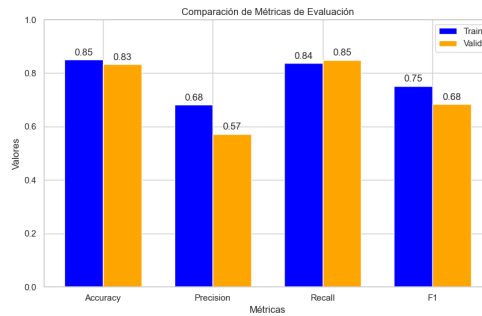


Figura 3.10: Metricas de evaluación del modelo de SVM para el conjunto de entrenamiento y validación

De igual manera, al inspeccionar las metricas del modelo podemos darnos cuenta que de hecho el comportamiento de los valores de dichas metricas es muy similar al modelo KNN, en ambos modelos podemos notar diferencias considerables entre el rendimiento mostrado en el conjunto de entrenamiento y validación, teniendo una variación de 0.07 en accuracy y 0.15 en la F1 score.

En cuanto a la curva roc podemos notar de igual manera un comportamiento similar, en la gráfica 3.13 podemos notar que en ambos modelos el comportamiento en el conjunto de entrenamiento es similar, aun que el clasificador SVM esta un poco por debajo del modelo KNN, de hecho es gracias a la AUC del modelo que podemos confirmar este hecho, ya que la AUC_{train} del modelo KNN es igual 0.96 mientras que AUC_{train} de SVM es igual a 0.93, ambos valores son muy buenos, sin embargo al comparar el rendimiento de dichos clasificadores en el conjunto de validación es donde podemos ver las diferencias entre ambos modelos. En este caso SVM no baja demasiado su rendimiento con respecto al conjunto de validación, podemos ver que la AUC_{valid} es igual

3. METODOLOGÍA DE ANÁLISIS Y PREPARACIÓN DE LOS DATOS

a 0.90, por lo cual la variación en el rendimiento entre conjunto de entrenamiento y validación es de 0.03.

Entonces podemos ver que a diferencia del modelo KNN, el clasificador SVM no presenta síntomas de sobre ajuste, a diferencia del modelo anterior, por lo cual SVM se perfila a ser un muy serio candidato a ser nuestro modelo final.

Finalmente analizaremos los resultados del modelo Random Forest, de forma parecida a los anteriores modelos podemos percatarnos que las matrices de confusión de los conjuntos de entrenamiento y validación (fig. 3.11) parecieran tener un rendimiento muy similar a los modelos anteriores, sin embargo mirando un poco más detenidamente nos daremos cuenta que de hecho su rendimiento mayor al de otros modelos como SVM y KNN, ya que el número de FN y FP es más bajo en comparación de los modelos ya mencionados.

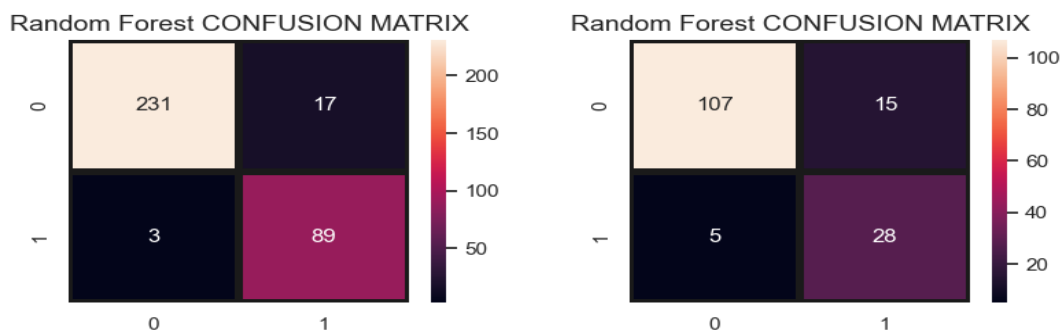


Figura 3.11: Matriz de confusión para el modelo random forest del conjunto de entrenamiento y validación

Este ultimo punto lo podemos reforzar revisando la gráfica 3.12, en la misma ver la alta efectividad de este modelo en el conjunto de entrenamiento, teniendo una accuracy de 0.94 y un F_1 de 0.9 siendo valores muy aceptables, sin embargo esta misma gráfica nos permite apreciar un gran inconveniente con este modelo, ese problema es el sobre ajuste, ya que en tanto la accuracy y el F_1 score tienen un bajo de rendimiento importante en el conjunto de validación, siendo de 0.7 y 0.16 para la accuracy y el F_1 respectivamente, lo cual claramente implica que este modelo no generalizara adecuadamente nuestra predicciones y no lo vuelve un mal candidato como modelo final a elegir.

3.4 Implementación de modelos de ML para predecir el abandono

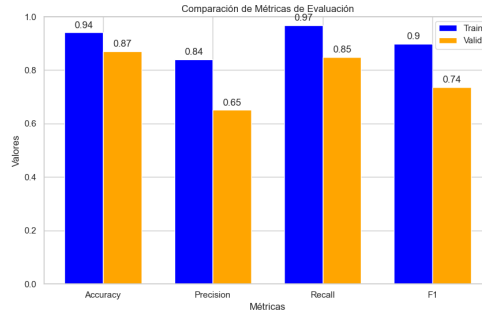


Figura 3.12: Metricas de evaluación del modelo de random forest para el conjunto de entrenamiento y validación

Recapitulando, hemos comparado distintos modelos aplicado las metricas anteriores, enfocándonos sobre todo en tener un buen F_1 -score y Precision, por ejemplo el modelo **deregresión logística** muestra un desempeño bajo, ya que no logra discriminar correctamente entre los clientes abandonadores y no abandonadores, por otro lado el modelo **KNN** tiene un mejor rendimiento que el modelo anterior, sufre de sobreajuste, lo cual nos lleva a concluir que no va a generalizar bien, por lo cual tenemos un potencial riesgo de obsolescencia en predicciones futuras, sin embargo en términos generales el modelo que cumple con el mejor desempeño general es el **SVM** esto debido a que tiene el mejor balance entre el conjunto de entrenamiento y validaciones, tiene mejor capacidad de generalización que el modelo anterior y por ultimo tenemos el modelo con el rendimiento mas alto en los datos de entrenamiento, pero en disminuye bastante su rendimiento en el conjunto de validación, lo cual es un claro ejemplo de sobre ajuste y por lo cual es un modelo que no generalizara bien.

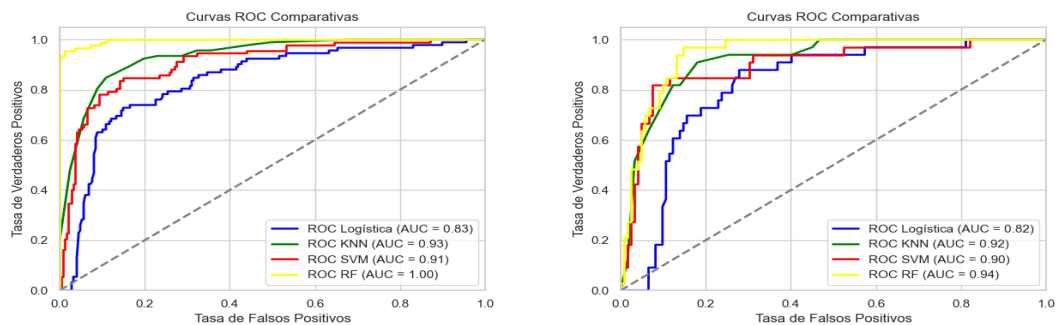


Figura 3.13: Curvas Roc de los modelos de clasificación para el conjunto de entrenamiento y validación respectivamente.

3.4.3. IA Explicable.

Dicho lo anterior, y con el propósito de interpretar de mejor manera el comportamiento de nuestro modelo final de clasificación, vamos a utilizar técnicas de **Explainable IA (XAI)**, todo esto apoyándonos de la librería SHAP, como dato curioso esta librería debe su nombre a **SHapley Additive exPlanations** que a su vez debe su nombre a Lloyd Stowell Shapley quien fuera un destacado matemático y economista estadounidense, y entre sus grandes aportes a la teoría de juegos y a la ciencia de datos se encuentran los valores de Shapley, los cuales son la base de los valores Shap. La idea es simple en ejecución, según la documentación de la librería "SHAP es un enfoque de teoría de juegos para explicar la salida de cualquier modelo de aprendizaje automático".

El propósito de esta metodología es poder dar una explicación clara de como los modelos muy complejos toman decisiones, en nuestro caso saber como el modelo de Support Vector Machine decide si un cliente es abandonador o no. Si bien nos podemos dar una idea acerca del funcionamiento de este modelo, usando SHAP podemos saber como cada característica impacta en las predicciones de nuestro modelo, de tal manera que "las características con valores SHAP positivos tienen un impacto positivo en la predicción, mientras que las que tienen valores negativos tienen un impacto negativo. La magnitud es una medida de cuán fuerte es el efecto." **cita**

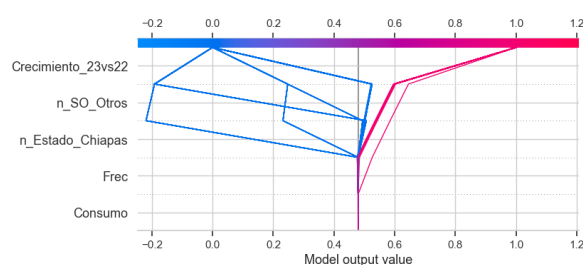


Figura 3.14: Gráfica de resumen global

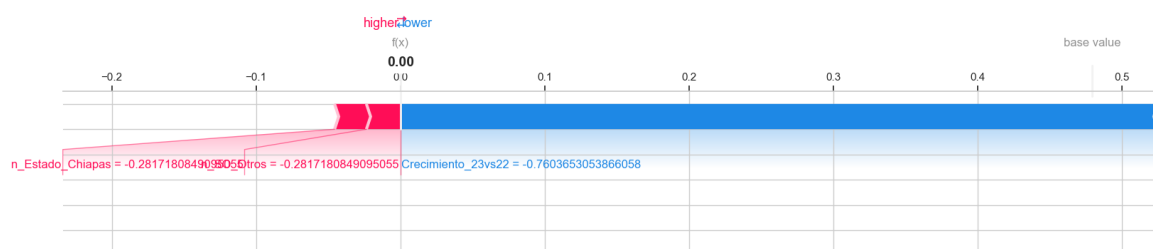


Figura 3.15: Gráfica de explicación local

3.4 Implementación de modelos de ML para predecir el abandono

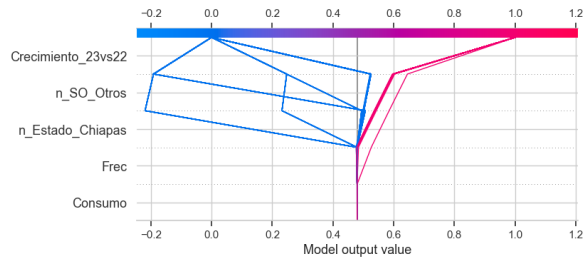


Figura 3.16: Gráfica de decisiones

Implementación y evaluación de modelos predictivos

4.1. Implementación de modelos de clasificación

4.1.1. Descripción de cada modelo y sus supuestos

4.2. Evaluación de desempeño de los modelos

4.2.1. Métricas: Accuracy, Precision, Recall, F1, AUC, PR

4.2.2. Comparación de resultados

4.3. Aplicación de XAI a modelos seleccionados

4.3.1. Interpretación de resultados y visualizaciones

Discusión y análisis de resultados

- 5.1. Discusión de hallazgos relevantes
- 5.2. Limitaciones del modelo
- 5.3. Recomendaciones para su implementación en entornos empresariales reales

Conclusiones y líneas futuras

6.1. Conclusiones generales

6.2. Aportaciones teóricas, metodológicas y prácticas

6.3. Sugerencias para investigaciones futuras

Código/Manuales/Publicaciones

A.1. Apéndice

Apéndice

Bibliografía

- Bruce, P., Bruce, A., and Gedeck, P. (2020). *Estadística Práctica para ciencia de datos con R y Python*. Anaya Multimedia. 23
- Chugh, V. (2023). Precision-recall curve in python tutorial. <https://www.datacamp.com/tutorial/precision-recall-curve-tutorial>. 40, 41
- Gold, C. (2020). *Fighting Churn with Data: The science and strategy of customer retention*. Manning. 25, 28, 29
- Géron, A. (2017). *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. O'Reilly. 5, 12, 39, 40
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Springer, New York, NY. 17
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444. 9
- Luger, G. F. (2005). *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*. Addison-Wesley, Boston, 5 edition. 8
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill Education. 5
- Peterson, M. (2023). *Marketing Analytics: Predicting Customer Churn in Python*. DataCamp. 38
- Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. Pearson, Boston, 4 edition. 8